SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

Code of Conduct:

I R.SHOBANA SHRI(1921260053) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *R.Shobana shri*

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*
Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5

3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5
4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5

8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5
9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5

13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5
15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Scenario-Based Assessment

***1.*An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.**

Fund Transfer Failures at Boundary Values: Test Design Analysis and Redesign

1. Critical Analysis of the Current Situation

The failures cluster at minimum and maximum limits. That pattern points to a design weakness, not bad luck.

Observation	What it implies
Failures at min/max limits	Boundaries were never tested deliberately
Team relies on random testing	Inputs are drawn from the whole range, so the edges are rarely hit
Failures are “frequent”	Defects are likely off-by-one errors (< vs <=) in validation code

Why random testing misses these defects: Suppose the valid range is ₹100 to ₹1,00,000, which is about 1,00,000 values. A random generator has roughly a 0.002% chance of picking exactly 100 or 1,00,000 in a single draw. The inputs that break the system are exactly the ones random testing is least likely to produce.

2. Assumed Specification

Since the problem gives no exact figures, I assume these for the design:

- Minimum transfer: ₹100
- Maximum transfer per transaction: ₹1,00,000
- Amount accepts up to 2 decimal places
- Source account balance must cover the amount

3. Equivalence Partitioning (EP)

I divide the amount input into classes that the system should treat the same way.

Partition	Range	Type	Expected result
EP1	Amount < 100 (including 0 and negatives)	Invalid	Rejected: “Below minimum limit”
EP2	100 ≤ Amount ≤ 1,00,000	Valid	Transfer succeeds
EP3	Amount > 1,00,000	Invalid	Rejected: “Exceeds maximum limit”
EP4	Non-numeric (letters, symbols, blank)	Invalid	Rejected: format error

EP5	Amount valid but > available balance	Invalid	Rejected: “Insufficient funds”

One representative per partition is enough to cover that class, which gives EP-based coverage with very few cases.

4. Boundary Value Analysis (BVA)

For each limit I test the value just below, at, and just above it, using the 3-value approach.

Lower boundary (100):

Test value	Expected
99.99	Reject
100.00	Accept
100.01	Accept

Upper boundary (1,00,000):

Test value	Expected
99,999.99	Accept
1,00,000.00	Accept
1,00,000.01	Reject

Extra edge values: 0, −1, 0.001 (precision overflow), and a very large number such as 99,99,99,999 (overflow check).

5. Redesigned Test Cases

TC ID	Technique	Input (₹)	Balance	Expected result
TC01	BVA	99.99	Sufficient	Rejected: below minimum
TC02	BVA	100.00	Sufficient	Success
TC03	BVA	100.01	Sufficient	Success
TC04	EP (valid)	50,000	Sufficient	Success
TC05	BVA	99,999.99	Sufficient	Success
TC06	BVA	1,00,000.00	Sufficient	Success
TC07	BVA	1,00,000.01	Sufficient	Rejected: above maximum
TC08	EP	0	Sufficient	Rejected
TC09	EP	−500	Sufficient	Rejected
TC10	EP	“abc” / “@#” / blank	Sufficient	Format error
TC11	EP/BVA	5,000	5,000 (exact)	Success, balance becomes 0
TC12	EP/BVA	5,000	4,999.99	Rejected: insufficient funds
TC13	BVA	100.005	Sufficient	Reject or round per rules
TC14	Error guess	1,00,000	Sufficient, but daily limit already used	Rejected: daily limit

TC11 and TC12 matter because a second boundary exists between the amount and the balance. Many teams test only the limit boundaries and miss this one.

6. Evaluating Random Testing

Criterion	Random testing	Verdict
Boundary defect detection	Very low	Weak
Repeatability	Poor unless the seed is logged	Weak
Traceability to requirements	None	Weak
Coverage guarantee	Can't be measured	Weak
Finds unexpected combinations	Sometimes	Strength
Low design effort	Yes	Strength

Decision: do not rely on it, but do not discard it entirely.

- Replace it as the primary strategy. For a financial system where errors cost money and trust, a technique with no coverage guarantee is unacceptable as the main method.
- Retain it in a supporting role. Run it as a smoke or fuzz layer after EP and BVA, to catch odd inputs the specification never anticipated (unusual characters, rapid repeated submissions, strange decimal formats).

7. Proposed Optimized Testing Approach

Layered strategy, in priority order:

1. **Requirement analysis:** Confirm exact limits, rounding rules, and daily and monthly caps with the business team. Ambiguity here causes many boundary bugs.
2. **EP:** Define the valid and invalid classes for amount, balance, account status, and transfer type.
3. **BVA**: Apply it to every numeric limit, including amount, balance, daily cap, and decimal precision.
4. **Decision table testing:** Cover combinations of conditions (balance sufficient or not, limit respected or not, account active or frozen, OTP valid or not) so that rule interactions are verified.
5. **State transition testing:** Cover the transaction lifecycle (initiated → authenticated → processing → success or failed or reversed), including timeouts and duplicate submissions.
6. **Error guessing**: Use past defect history, such as network drops mid-transfer, double clicks, and session expiry.
7. **Controlled random/fuzz testing:** Apply seeded, logged random inputs as a final safety net.
8. **Automation:** Put all BVA and EP cases into a data-driven regression suite that runs on every build.

8. Justification

- Defect detection: BVA targets the exact place where the failures occur. Off-by-one errors in conditions like amount > 100 instead of amount >= 100 are caught by TC01 to TC03 alone.
- Efficiency: EP cuts thousands of possible inputs to a handful of representatives, with no loss of class coverage.
- Traceability: Every test case maps to a requirement and a technique, so coverage can be shown to auditors.
- Risk-based: Effort is concentrated on high-risk areas (limits, balances, concurrency), which suits banking.

- Repeatability: The suite is deterministic and automatable, so a defect found once is guarded against forever.

9. Expected Outcome

Metric	Before (random-led)	After (EP + BVA-led)
Boundary defect detection	Low, accidental	High, deliberate
Test cases needed	Large, uncertain	About 14 core cases per feature
Reproducibility	Poor	Full
Production boundary failures	Frequent	Significantly reduced

10. Conclusion

The failures happen because the existing strategy never targets boundaries. I recommend making EP and BVA the foundation, supporting them with decision tables, state transitions, and error guessing, and keeping random testing only as a seeded, secondary fuzz layer. This gives structured coverage, early detection of limit-related defects, and a regression suite that keeps those defects from returning.

2.A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.

White Box Testing of a Healthcare Module: Coverage Gaps and a Redesigned Strategy

1. Evaluation of the Testing Approach Used

Critical defects escaped because the suite exercised only the common path (valid adult, valid dose) and ignored the clinically risky logic. The visible symptoms are:

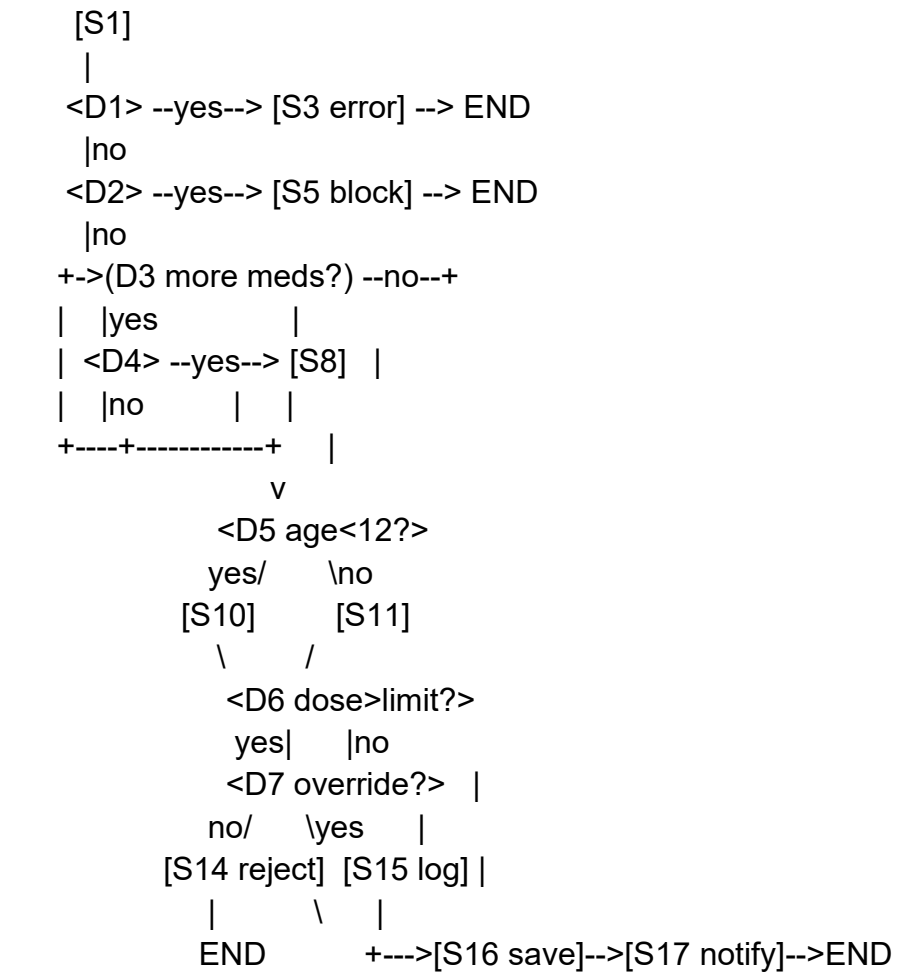
- **Tests were written from the "what should work" viewpoint.** Nobody asked which statements and decisions had never executed.
- **No coverage target was set.** Without a measurable adequacy criterion, "enough testing" was a judgment call.
- **Safety branches were treated as rare.** Allergy blocks, pediatric dosing, and overdose overrides are rare in test data but frequent in real patient populations.
- **Loops and exception paths were not designed for.** The tests cover "one medication" and never "none" or "many".

2. Conceptual Module Under Analysis

I model a Prescription Validation module, a typical high-risk healthcare component. This is a conceptual example. The same method applies to any module.

```
S1   read patient, drug, dose
D1   if patient not found          -> S3 return error
D2   if patient allergic to drug   -> S5 block and return
D3   for each current medication:
D4      if interacts(med, drug)    -> S8 flag interaction
D5   if age < 12                   -> S10 limit = weight-based
     else                          -> S11 limit = standard
D6   if dose > limit
D7      if override not authorized -> S14 reject
        else                       -> S15 log override
S16  save prescription
S17  notify pharmacy
```

3. Control Flow Graph (Conceptual)



Cyclomatic complexity: V(G) = predicate nodes + 1 = 7 + 1 = 8

So at least 8 independent paths must be exercised. Any suite with fewer than 8 tests cannot achieve basis-path coverage.

4. Assessment of Coverage Achieved

Assume the original suite had three tests: T1 (valid adult, one non-interacting med, normal dose), T2 (patient not found), T3 (allergy block).

Criterion	Covered	Total	Achieved
Statement	12 (S1–S7, S9, S11, S12, S16, S17)	17	**≈ 71%**
Branch (decision outcomes)	8	14	**≈ 57%**
Independent (basis) paths	3	8	**≈ 38%**

Missing outcomes: loop skipped (D3), interaction found (D4), age < 12 (D5), dose exceeds limit (D6), and both outcomes of D7.

Key insight: 71% statement coverage looks acceptable on a dashboard, yet the uncovered 29% is exactly the pediatric dosing, interaction flagging, and overdose handling logic. Statement coverage is the weakest criterion and hides the most dangerous gaps. Because one test can touch many statements without testing any decision's false side, it rewards the happy path.

5. Missing Paths Identified

Path	Conditions	Clinical risk if untested
P4	Patient has no current meds (loop skipped)	Null or empty list crash
P5	Interaction detected	Dangerous drug combination goes unflagged
P6	Age < 12	Adult dose given to a child
P7	Dose over limit, no override	Overdose saved
P8	Dose over limit, override authorized	Audit trail missing
Implicit	Save or notification failure	Prescription saved but pharmacy never told

6. Improved Testing Strategy

I use layered criteria, each closing a specific gap.

Layer 1: Basis path testing (8 tests, one per independent path)

TC	Scenario	Decisions exercised
TC1	Patient not found	D1 = T
TC2	Allergic to drug	D1 = F, D2 = T
TC3	Adult, one med, no interaction, dose OK (baseline)	D3 enter, D4 = F, D5 = F, D6 = F
TC4	Adult, no current meds	D3 skipped
TC5	Adult, interacting med	D4 = T
TC6	Child (age < 12), dose within limit	D5 = T
TC7	Dose over limit, no override	D6 = T, D7 = no
TC8	Dose over limit, override authorized	D7 = yes

This achieves 100% statement, 100% branch, and 100% basis-path coverage with a minimal suite.

Layer 2: Condition and boundary refinement

- Test D5 at ages 11, 12, and 13, and D6 at dose = limit, limit + 0.01, and limit − 0.01. This catches < versus <= mistakes that branch coverage alone cannot see.
- If any decision becomes compound (for example, allergic OR cross-reactive), apply MC/DC so each condition is shown to independently affect the outcome.

Layer 3: Loop testing for D3

Test 0, 1, 2, a typical number (5), and the maximum supported medications. This covers skip, single-pass, repeat, and capacity behavior.

Layer 4: Data flow testing

Track the definition and use of limit, dose, and the interaction flag. Every definition (S10, S11) must reach a use (D6). This catches cases like a flag set in S8 but never consulted before saving.

Layer 5: Exception path tests

Inject faults for database save failure and pharmacy notification failure. Verify rollback or retry so a prescription is never half-committed.

Layer 6: Test-suite quality check

Run **mutation testing** (flip > to >=, delete S8, swap S10 and S11). Surviving mutants show where assertions are too weak, since coverage proves execution but not verification.

7. Justification of Decisions

- **Why basis path rather than full path coverage:** Full path coverage is infeasible. The loop at D3 makes the number of paths unbounded, and even with the loop capped at a few iterations the path count multiplies across the remaining decisions. Basis path testing gives a provably sufficient, finite set of 8 tests that covers every independent route.

- **Why branch coverage is the mandatory minimum:** It forces both sides of every decision to run. For a healthcare system, an untested “false” side is an untested safety rule.

- **Why add boundaries and MC/DC on top:** Structural coverage can reach 100% while still missing a wrong comparison operator. These layers target that residual risk.

- **Why mutation testing:** It measures whether the tests would actually fail when the code is wrong, which coverage metrics cannot show.

- **Why exception paths:** The most damaging real-world defects often sit in failure handling, not in the main logic.

Realistic target: 100% statement and branch, 100% basis path, MC/DC on all compound safety conditions, and a mutation score above roughly 85%. Anything below full branch coverage should block release for safety-critical modules.

8. How This Improves Reliability and Reduces Risk

Aspect	Before	After
Statement coverage	≈ 71%	100%
Branch coverage	≈ 57%	100%
Basis paths tested	3 of 8	8 of 8
Safety-critical logic (pediatric, interaction, overdose)	Untested	Verified explicitly
Failure handling	Unverified	Fault-injection tested
Test effectiveness	Unmeasured	Quantified via mutation score

- **Reliability:** Every decision and independent route has been executed and verified, so behavior is predictable across patient types.
- **Patient safety:** The risky branches are tested before deployment instead of being discovered by patients.
- **Regression protection:** The suite, tied to the graph, runs automatically in CI. Any new decision added to the code raises V(G) and demands a new test.
- **Compliance and traceability:** Coverage reports give auditors measurable evidence of adequacy, which regulated healthcare software requires.

9. Conclusion

The post-deployment failures came from judging adequacy by test count and a flattering statement-coverage figure. The control flow graph shows 8 independent paths, of which the old suite covered 3, and the missing five hold the clinically critical logic. Replacing ad hoc testing with basis path testing, supported by boundary and MC/DC checks, loop and data flow testing, fault injection, and mutation analysis, gives complete and measurable coverage where it matters and makes the module both more reliable and far less risky to deploy.

3.A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.

Multi-Field Input Defects in a Retail E-Commerce System: Black Box Redesign

1. Scenario Analysis

Price, quantity, and discount each seem to work when tested alone, yet errors appear for specific combinations. That points to **interaction defects**: the fault depends on two or more fields together, so single-field checks cannot expose it.

Likely sources in this system:

Source	Example
Derived-value rules	Order cap checked on price × qty instead of the discounted total
Conditional eligibility	Discount allowed only above a minimum price
Rounding order	Discount applied before or after rounding to 2 decimals
Validation order	Quantity checked before price, so the second error is masked
Default masking	Testers leave other fields at defaults (qty = 1, discount = 0), hiding the interaction

2. Assumed Specification

The problem gives no limits, so I assume these:

- **Price:** ₹0.01 to ₹99,999.99, 2 decimals
- **Quantity:** integer, 1 to 50
- **Discount:** 0% to 50%, 2 decimals
- **Rule R-A:** a discount is allowed only if the unit price is ≥ ₹100
- **Rule R-B:** the final payable total (after discount) must not exceed ₹5,00,000
- **Total** = price × qty × (1 − discount/100), rounded to 2 decimals

3. Limitations of the Current Approach

Limitation	Consequence
Fields tested one at a time	Interactions never exercised
Other fields kept at “safe” defaults	Combined boundary cases never occur
Only inputs were checked, not the computed total	Rule R-B violations go unseen
No rule-based model	Eligibility conditions were never enumerated
No combination selection method	Coverage is accidental

Scale of the problem: The raw input space is about 10^7 prices × 50 quantities × 5,001 discount values ≈ **2.5×10^{12}** combinations. Exhaustive testing is impossible, so the goal is *systematic* completeness: every class, every boundary, every rule, and every pair of values is covered.

4. Equivalence Class Partitioning

Field	Valid classes	Invalid classes
Price	P1: 0.01–99.99 (not promo-eligible); P2: 100–99,999.99 (promo-eligible)	P3: ≤ 0; P4: > 99,999.99; P5: non-numeric/blank
Quantity	Q1: 1–50	Q2: ≤ 0; Q3: > 50; Q4: decimal; Q5: non-numeric/blank
Discount	D1: 0; D2: 0.01–50	D3: < 0; D4: > 50; D5: non-numeric/blank

Splitting the valid price range at ₹100 is deliberate. The rule R-A creates a hidden partition boundary that a naive range of "0.01 to 99,999.99" would miss.

5. Boundary Value Analysis

Field	Boundary test values
Price	0, 0.01, 0.02, 99.99, 100.00, 100.01, 99,999.99, 100,000.00
Quantity	0, 1, 2, 49, 50, 51
Discount	−0.01, 0, 0.01, 49.99, 50, 50.01
Computed total	4,99,999.99, 5,00,000.00, 5,00,000.01

The last row tests a derived boundary. It is not an input at all, so field-level BVA never reaches it. It is reached by choosing inputs that land the total on the cap.

6. Decision Table Testing

Conditions apply to inputs that are individually valid. C1: unit price ≥ 100. C2: discount > 0. C3: discounted total ≤ ₹5,00,000.

Rule	C1	C2	C3	Expected action
R1	N	N	Y	Accept, full price
R2	N	Y	Y	Reject discount (not eligible)
R3	N	–	N	**Infeasible** (max 99.99 × 50 = 4,999.50)
R4	Y	N	Y	Accept, full price
R5	Y	N	N	Reject (cap exceeded)
R6	Y	Y	Y	Accept, discounted price
R7	Y	Y	N	Reject (cap exceeded after discount)

Six feasible rules. Eliminating R3 shows the table reasoning prevents wasted tests.

Test cases derived from the table, with boundary tightening:

TC	Price	Qty	Disc %	Rule	Expected
DT1	50.00	2	0	R1	Total 100.00
DT2	99.99	1	10	R2	Discount rejected
DT3	100.00	1	10	R6	Total 90.00
DT4	10,000.00	50	0	R4	Accepted (exactly 5,00,000)
DT5	10,000.01	50	0	R5	Rejected (5,00,000.50)
DT6	11,000.00	50	10	R6	Accepted (4,95,000), proving the cap uses the *discounted* total
DT7	12,000.00	50	10	R7	Rejected (5,40,000)
DT8	100.05	1	0.5	R6	Total 99.55 (rounded), eligibility judged on *pre-discount* price

DT6 and DT8 are the likeliest to reveal what the current testing missed: **the order of operations**.

7. Pairwise (All-Pairs) Testing

For valid-value combinations I select representative values:

- Price: {0.01, 99.99, 100, 50,000, 99,999.99}
- Quantity: {1, 2, 50}
- Discount: {0, 0.01, 25, 50}

Full combination: 5 × 3 × 4 = **60 tests**.

A pairwise generator (PICT or AllPairs) covers all 47 value pairs in about **20 tests**, since the minimum is bounded by the two largest factors (5 × 4).

Important limit: Rule R-B depends on all *three* fields at once. Pairwise guarantees only two-way coverage, so it cannot be relied on for that rule. The decision table covers it, and pairwise covers the remaining, less structured interactions. Each technique compensates for the other's blind spot.

8. Other Black Box Methods Applied

- **Cause-effect graphing:** Used to derive the decision table above and to expose the infeasible rule R3.
- **Error guessing:** Rounding (33.33% on 3 units), very small totals, max price with a 50% discount, pasted values with spaces or commas, and rapid repeated edits of one field.
- **State/sequence checks:** Change discount after quantity is entered, then change quantity again, to confirm the total recalculates and stale values do not persist.

9. Revised Test Design Strategy

Layer	Technique	Purpose	Approx. tests
A	Invalid-class isolation (one invalid field, others valid)	Every error message reachable and not masked	14
B	Valid-edge BVA	Individual limits	8
C	Decision table + derived boundaries	Rule logic and the total cap	10
D	Pairwise on valid values	Two-way interactions	20
E	Error guessing and recalculation sequences	Rounding, ordering, UI behavior	6
	Total		**≈ 58** (versus 2.5×10^{12} exhaustive)

Design rule: In Layer A, only one field is invalid at a time. Combining two invalid inputs can hide one of the defects behind the other's error message.

10. Justification

- **Defect detection:** Interaction faults are targeted directly. Pairwise surveys show that most combination defects involve only two or three factors, which is the reason this mix works.
- **Test completeness:** Completeness is defined by measurable criteria: 100% of equivalence classes, all boundaries, all feasible decision-table rules, and all two-way value pairs.
- **Efficiency:** About 58 tests replace an impossible exhaustive set, with each test traceable to a class, boundary, rule, or pair.
- **Honesty about limits:** The suite does not prove the absence of 4-way or deeper interactions. Because only three fields interact here, the risk is small, and the table covers the one known three-way rule.
- **Maintainability:** When a new field is added (for example, a coupon code), the decision table gains a condition and the pairwise model gains a factor, so the suite grows in a controlled way.

11. Expected Improvement

Measure	Before	After
Interaction defects found	Few, by chance	Targeted by table and pairwise
Derived-value rules tested	None	R-A and R-B at their boundaries
Rounding and order-of-operations faults	Unchecked	Explicit cases (DT6, DT8)
Coverage claim	"We tested many cases"	Measurable per criterion
Reproducibility	Ad hoc	Fully data-driven and automatable

12. Conclusion

The testers missed errors because they validated each field separately, while the defects lived in the relationships between price, quantity, and discount. Combining equivalence partitioning and boundary analysis for individual fields, a decision table for the business rules, pairwise testing for broad interaction coverage, and targeted error guessing for rounding and sequencing gives complete coverage of every class, boundary, rule, and pair in about 58 tests. That is far stronger defect detection than random combinations, at a size that can run on every build.

4.A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.

Path Testing of a Module with Nested Conditions and Loops

1. Situation Analysis

The team reports only "partial path coverage" but does not say which paths are missing or why. Nested decisions and loops make the path count grow combinatorially, so teams under time pressure test a few intuitive routes and stop. The untested routes are usually the ones that hold the defects.

Conceptual module used for the analysis (batch transaction settlement):

```
1   total = 0; errors = 0; i = 0
2   while (i < n)                       // P1
3     if (txn[i].valid)                 // P2
4       if (txn[i].type == CREDIT)      // P3
5         if (amt > limit) flag(txn[i])   // P4
6         total += amt
7       else
8         if (balance < amt)            // P5
9           reject(txn[i]); continue    // note: skips i++
10        total -= amt
11    else
12      log(txn[i]); errors++
13      if (errors > 3) break           // P6
14    i++
15  if (total < 0 && errors == 0)       // P7 && P8
16    raiseAlert()
17  return total
```

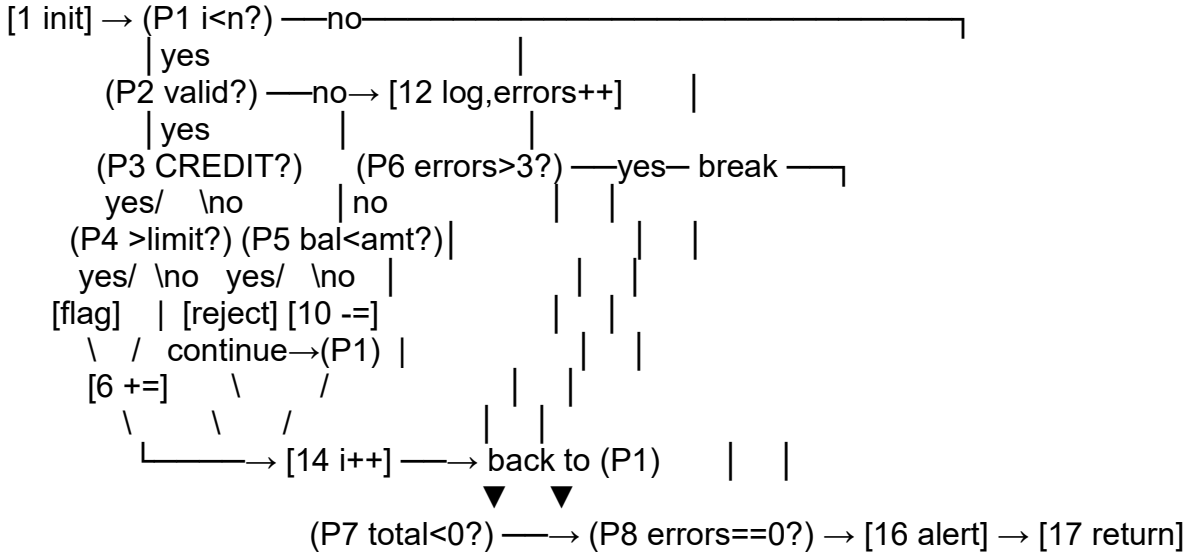
The code has one loop, five nested decisions, two loop-exit jumps (continue and break), and one compound condition.

2. Gaps in Control Flow Graph Construction

A path analysis is only as good as the graph behind it. These are the construction mistakes that typically lead to partial coverage.

Gap	What goes wrong here	Effect
continue drawn as a normal edge	The edge from line 9 should jump to the loop test and bypass node 14	The index never advances, so an infinite loop goes unnoticed
break edge omitted	The exit from line 13 to line 15 is missing	The "four bad transactions" path is never planned
Compound condition as one node	total < 0 && errors == 0 is drawn as a single predicate	The case "total negative but errors present" is never designed
Loop back-edge merged	Iteration 1 and iteration 2 are treated as identical	Behavior that depends on state from earlier iterations is missed
No join nodes after nested ifs	Branches merge ambiguously	Paths are miscounted and the complexity figure is wrong
Single exit assumed	Early exits are ignored	Exit-dependent state (such as errors) is not checked

3. Corrected Control Flow Graph



4. Complexity and Independent Path Count

Counting each simple condition as its own predicate: P1 to P8 gives 8 predicates.
V(G) = 8 + 1 = 9, so at least 9 independent paths must be tested.

The total number of distinct paths is much larger. Each loop iteration can end in about 5 routes (credit within limit, credit over limit, debit accepted, debit rejected, invalid). With the loop bounded to 3 iterations, that is on the order of $5^3 + 5^2 + 5 + 1$, well over **150 paths**. This shows why exhaustive path testing is impossible and why selecting independent and critical paths is the right goal.

5. Assessment of Current Coverage

Assume the team's tests cover four routes: a clean credit, a credit over the limit, a debit that leaves a negative total, and one invalid transaction.

Criterion	Achieved	Comment
Statement	≈ 88%	Looks acceptable on a report
Branch	13 of 16 outcomes ≈ 81%	Missing: reject branch, break branch, errors != 0 outcome
Basis paths	4 of 9 ≈ **44%**	The true measure of adequacy here
Loop boundaries	1 of 7	Only a one-iteration case was tried

The high statement figure hides the problem. The uncovered 12% contains the rejection logic and the error-limit exit, which are the places where the real faults live.

6. Impact of Inadequate Independent Path Identification

Missed path	Defect it would have exposed	Severity
Debit rejected (line 9)	continue skips i++, causing an infinite loop or repeated rejection	Critical (system hang)
Four invalid transactions → break	errors is incremented after the check, so the break fires one transaction late or early	High
Negative total with errors > 0	Alert wrongly raised or wrongly suppressed because of the && logic	High
Loop skipped (n = 0)	Returns an uninitialized or stale total	Medium
Rejected debit followed by more transactions	Later transactions are skipped or double-processed	High

Each of these faults needs a specific combination of decisions. Because tests were chosen by intuition and not derived from independent paths, there was no systematic way to know a combination was absent.

7. Complete Path Testing Strategy

Layer 1: Basis path set (9 tests, mandatory)

BP	Scenario	Key conditions
BP1	n = 0, loop skipped	P1 = F immediately
BP2	One valid credit within limit (baseline)	P2 T, P3 T, P4 F
BP3	Credit above limit	P4 T
BP4	Debit accepted, total ends negative, no errors	P5 F, P7 T, P8 T → alert
BP5	Debit rejected as the only transaction	P5 T, continue edge
BP6	One invalid transaction	P2 F, P6 F
BP7	Four invalid transactions	P6 T → break
BP8	Negative total after an invalid transaction	P7 T, P8 F → no alert
BP9	Rejected debit followed by a valid credit	Proves the index advances after continue

Layer 2: Loop testing

Test n = 0, 1, 2, a typical value (m), n − 1, n, and n + 1 where n is the supported maximum batch size. This covers skip, single pass, repeated passes, and overflow of the capacity limit.

Layer 3: MC/DC for the compound condition (line 15)
Three tests (n + 1 for n = 2 conditions) show that each condition independently flips the outcome:

Test	total < 0	errors == 0	Alert
M1	T	T	Yes
M2	T	F	No
M3	F	T	No

Layer 4: Critical path selection by risk
Add multi-iteration paths for the combinations where state carries across iterations:

- Credit → rejected debit → credit (state after continue)
- Invalid ×3 → valid debit → invalid (the fourth error arrives after valid activity)
- Debit → debit → debit pushing the total further negative (alert logic with several decrements)

Layer 5: Data flow testing
Cover every definition-use pair of total, errors, and i. For example, the definition of errors on line 12 must reach its use on lines 13 and 15. The i++ on line 14 must be reachable after every non-exit branch, which would have flagged the continue defect directly.

Layer 6: Feasibility check and mutation testing

- Mark infeasible paths (for example, “debit rejected” when the batch contains only credits) so they are excluded from the target and documented, not silently dropped.
- Apply mutations (> to >=, remove i++, swap && for ||, change 3 to 4) and require that the suite kills at least 85% of them.

8. Justification Using Coverage Criteria

Criterion	Role in strategy	Reason
Statement	Baseline only	Cheap but blind to untested decision outcomes
Branch	Mandatory release gate (100%)	Forces both sides of every decision to run
Basis path	Primary adequacy measure	Finite (9), provably sufficient for independent routes
Loop boundary	Required	Defects at 0, 1, and n + 1 iterations are common and not guaranteed by branch coverage
MC/DC	Required for compound conditions	Shows each condition matters, not just the combined result
All-uses data flow	Supporting	Finds stale or unused variable states
Full path	Not attempted	Unbounded because of the loop, and not cost-justified

The decision to use basis paths with risk-selected extras follows a clear rule: cover every independent route for completeness, then spend remaining effort only on paths where state accumulates across iterations, since that is where nested loop defects hide.

9. Expected Outcomes

Measure	Before	After
Statement coverage	≈ 88%	100%
Branch coverage	≈ 81%	100%
Basis path coverage	4 of 9	9 of 9
Loop boundary cases	1 of 7	7 of 7
Compound condition verified	No	MC/DC met

Hang and early-exit defects	Undetected	Caught by BP5, BP7, BP9
Suite size	Ad hoc	About 22 targeted tests (9 basis + 7 loop + 3 MC/DC + 3 risk paths, with overlap removed)
Effectiveness evidence	None	Mutation score ≥ 85%

10. Conclusion

The partial coverage was a consequence of a flawed graph and an unmeasured selection of paths. Jumps like continue and break were not modeled, the compound condition was treated as a single decision, and independent paths were never counted. Rebuilding the graph, computing V(G) = 9, and deriving the nine basis paths gives a finite and complete route set. Loop boundary tests, MC/DC, risk-selected multi-iteration paths, data flow checks, and mutation analysis then close the remaining gaps. This ensures that every critical path, including the ones that would hang the system or suppress an alert, is exercised and verified before release.

5.A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.

Hybrid Testing for a Financial Application: Why Two Techniques Still Leaked Defects

1. Core Diagnosis

Using both black box and white box testing does not make a strategy hybrid. The likeliest cause of the production defects is that the two were run as **separate, parallel activities**: functional testers designed from the specification and never looked at the code, while developers wrote unit tests from the code and never looked at the specification. Each group then reported its own numbers, and nobody asked what *neither* group had covered.

Each side has a blind spot the other would have covered:

Approach	Finds	Cannot find
Black box (ECP, BVA)	Missing or misread requirements, wrong limits	Code that exists but is not described in the specification (hidden branches, defensive checks, legacy rules)
White box (CFG, paths)	Untested logic, unreachable code, wrong operators	Requirements that were never implemented, because there is no code to cover

A defect that falls into the overlap of both blind spots escapes both teams. Examples are a spec rule that was implemented in the wrong place, and an undocumented branch whose trigger value sits on a boundary nobody derived.

2. Analysis of the Integration Failures

Failure	How it appears in a financial system	Consequence
No shared traceability	Test cases are not linked to both a requirement and a code element	Nobody can see which requirements lack structural evidence, or which code lacks a requirement
Black box suite not measured structurally	Functional tests ran but code coverage was never collected during them	Unknown how much of the code the specification-based tests really reach
White box suite not checked against the spec	Unit tests assert what the code does, not what it should do	A wrong rule can be 100% covered and still wrong
Boundaries designed only from the spec	BVA used the documented limit (say ₹1,00,000) but the code also has internal thresholds such as a fraud-review cutoff or a rounding-based comparison	Internal boundaries untested
Same data for everything	Tests run on clean, small, single-currency data	Defects tied to volume, precision, and concurrency remain hidden
Coverage as a target	"80% coverage" accepted as a release criterion	Executed code is mistaken for verified code
No feedback loop	Production defects are fixed but not analyzed for the technique that should have caught them	The same class of defect returns

3. Evaluation of Adequacy Criteria and Metrics

Typical misuse, and the correction for each:

Metric as used	Problem	Correct use
Statement coverage as the headline	Rewards the happy path, ignores false branches	Use only as a floor
Branch coverage reported on the unit suite only	Says nothing about whether the spec-derived tests reach the same branches	Measure it per suite and combined
Path coverage claimed without a CFG	Not credible; loops make full path coverage unbounded	Compute V(G) and cover basis paths
ECP counted as "classes tested"	Valid classes covered, invalid classes and derived-value classes skipped	Require valid and invalid classes for every input, plus classes from internal rules
BVA on inputs only	Computed values (fees, interest, running balance) have boundaries too	Apply BVA to outputs and intermediate values
No effectiveness measure	Coverage shows execution, not detection	Add mutation score and defect-escape rate

Principle: coverage shows what has *not* been tested. It never shows that what has been executed was *verified*. A financial module can reach 100% branch coverage with assertions so weak that a changed > to >= still passes.

4. Proposed Hybrid Strategy: "Spec-First, Structure-Checked"

The sequence matters. Black box defines *what must be true*. White box then audits *whether the code was exercised in every way that matters*, and the gaps feed back into new tests.

Stage 1: Requirement analysis and traceability
Assign every requirement an ID. Every test case will later carry the requirement ID, the technique, and, once measured, the code elements it touches. This single matrix ties both sides together.

Stage 2: Black box design (specification view)
ECP, with an illustration on a funds-transfer function whose spec says: amount ₹100 to ₹1,00,000, balance must cover the amount and fee, fee is 0.5% rounded to 2 decimals.

Input	Valid classes	Invalid classes
Amount	100 to 1,00,000	below 100, above 1,00,000, zero or negative, non-numeric
Balance vs (amount + fee)	sufficient	insufficient
Account status	active	frozen, closed, dormant

BVA, applied to inputs and to derived values:

Target	Values
Amount, lower	99.99, 100.00, 100.01
Amount, upper	99,999.99, 1,00,000.00, 1,00,000.01
Balance against amount + fee	balance equal to total, one paisa short
Fee rounding	amounts producing x.xx5 fees (for example ₹100.10 gives a fee of 0.5005)

Decision table for rule combinations such as account status × balance × limit status, since financial rules are mostly conjunctions.

Stage 3: Run the black box suite under coverage instrumentation
Execute the spec-derived tests with a coverage tool attached. This is the point at which the two approaches genuinely meet. Record statement, branch, and condition coverage achieved by the functional suite alone.

Stage 4: White box audit (structure view)
1. Build the CFG for each critical function (transfer, interest accrual, reconciliation, fee calculation). Model break, continue, early returns, exception edges, and split compound conditions into separate predicate nodes.
2. Compute V(G) to obtain the minimum number of independent paths.
3. Overlay the Stage 3 results on the graph and mark covered and uncovered edges.
4. Classify every uncovered element with this rule:

Uncovered element is...	Meaning	Action
Reachable and linked to a requirement	Black box design gap	Add ECP/BVA cases (a missed class or boundary)
Reachable but linked to no requirement	Undocumented behavior	Raise with the business analyst: confirm the rule or remove the code
Unreachable	Dead code or infeasible path	Document and remove or justify
Exception or failure handler	Neither side designed for it	Add fault-injection tests

Step 4 turns coverage from a score into a defect-finding tool.

Stage 5: Targeted white box additions

- Basis path tests for each unmet independent path.
- Internal boundary tests for thresholds found in the code but absent from the spec (for example, an if (amt >= 200000) fraud-review rule).
- Loop tests (0, 1, 2, typical, max, max + 1) for batch and statement processing.
- MC/DC for compound safety conditions, such as isActive && !isFrozen && withinLimit.
- Data flow tests for definition-use pairs of balance, fee, and running totals.

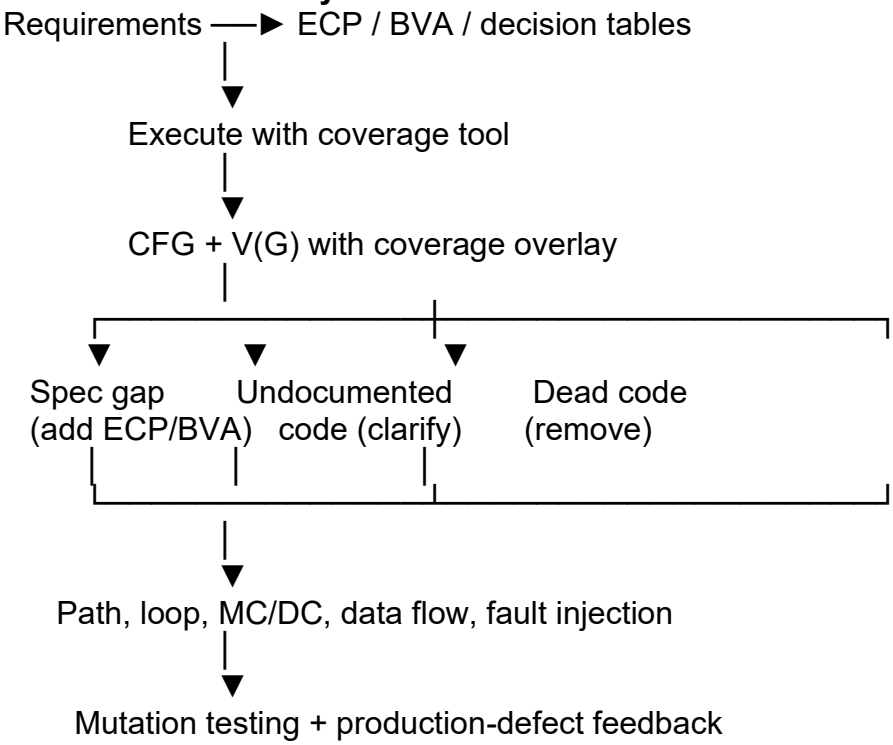
Stage 6: Financial-specific risk layers

Risk	Technique
Rounding and precision	Fixed-point or decimal-type checks; sums of many rounded values against a total
Concurrency (double debit)	Parallel submissions of the same transfer
Idempotency and retries	Replay a request after a timeout
Partial failure	Fault injection between debit and credit; verify atomic rollback
Reconciliation	End-of-day totals must equal the sum of transactions
Audit trail	Every state change logged

Stage 7: Effectiveness verification

- **Mutation testing on core financial functions:** flip operators, shift constants, delete statements. Target a mutation score of about 85% or higher on critical modules.
- **Defect-escape analysis:** for each production defect, record which technique should have caught it and why it did not. Feed the answer back to Stages 2 to 5.

5. Workflow Summary



6. Release Criteria

Criterion	Target	Applies to
Requirement-to-test traceability	100%	All requirements
Equivalence classes (valid and invalid)	100%	All inputs
Boundaries (input, derived, internal)	100%	Money-handling fields
Branch coverage (combined suites)	100% on critical modules, at least 90% elsewhere	Financial logic
Basis path coverage	100% of V(G)	Critical modules
MC/DC	Met	Compound authorization conditions
Uncovered code	Each item classified and resolved	Whole module
Mutation score	At least 85%	Critical modules
Open exception-path tests	None	Transfers, settlement

Any critical-module miss blocks release. Non-critical modules may justify deviations in writing.

7. Justification: How This Gives Better Coverage and Prevention

Concern	How the hybrid addresses it
Missed requirements	Black box design starts from the spec, so unimplemented or misread rules fail a test
Hidden code behavior	The CFG audit exposes branches and thresholds the spec never mentions
Overlap blind spots	The classification rule in Stage 4 forces every uncovered element to be explained
False confidence from metrics	Several criteria are combined, and mutation scoring checks that tests actually detect faults
Repeat defects	Escape analysis closes the learning loop
Audit and regulation	Traceability plus measurable criteria give evidence of adequacy
Cost	Black box gives broad coverage cheaply; white box effort is spent only on measured gaps, so there is no blind duplication

Why the order is specification first and structure second: reversing it lets existing code define "correct", which preserves bugs. Measuring structure after spec-based testing shows precisely what the specification-driven design failed to reach.

8. Limitations to Acknowledge

- Coverage of 100% does not prove correctness, because a missing feature leaves no code to cover.
- Basis paths cover independent routes, not every combination; risk-based extra paths are selected deliberately.
- Mutation testing is computationally expensive, so it is reserved for critical modules.
- The approach depends on an accurate specification. Ambiguous requirements must be resolved first, or the test oracle itself will be wrong.

9. Conclusion

Production defects persisted because the black box and white box efforts ran side by side without ever being reconciled, and because coverage percentages were treated as proof of quality. The improved strategy designs tests from the specification using ECP, BVA, and decision tables, runs them under instrumentation, builds the CFG and computes V(G), and then classifies every

uncovered element as a design gap, an undocumented rule, or dead code. Path, loop, MC/DC, data flow, and fault-injection tests close the structural gaps. Mutation testing and defect-escape analysis confirm the suite detects faults and improves it over time. The two techniques become one feedback system, which gives measurable coverage and a much lower chance that a financial defect reaches production.

6.A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.

Payroll Bonus Failures at 5, 10 and 15 Years: Analysis and BVA/ECP Redesign

1. Critical Analysis of the Failure
Incorrect payouts at exactly 5, 10 and 15 years of service point to one defect class: tier-boundary comparison errors. The faulty employees sit on the edge where one bonus tier ends and the next begins, so the code almost certainly places them in the wrong tier.

Observation	Probable cause
Failures only at exact multiples of 5	Operator error: years > 5 written where the spec says years >= 5 (or the reverse)
All three boundaries fail, not just one	The same mistake repeated across a tier table or copy-pasted conditions
Other service values pay correctly	The logic inside each tier is right; only the **edges** are wrong
Defect reached testers late	Test data never contained employees at exactly these values

Assumed specification (the problem gives no figures, so I fix these to make the design concrete):

Service tier	Years of service (completed)	Bonus multiplier base
T0	0 to under 5	2% of monthly salary
T1	5 to under 10	5%
T2	10 to under 15	8%
T3	15 and above	12%

Performance rating (1 to 5) adjusts the bonus: rating 1 gives 0×, rating 2 gives 0.5×, rating 3 gives 1.0×, rating 4 gives 1.25×, rating 5 gives 1.5×.

Formula: Bonus = Salary × tier % × rating factor.
An employee with exactly 5 years should land in T1 (5%). A years > 5 bug sends them to T0 (2%). The payout is wrong by a factor of 2.5, and across 10 and 15 years the error repeats. These are exactly the symptoms reported.

Additional likely contributors beyond the operator:

- **Definition of "years"**: whole years completed versus a fractional calculation from the join date. An employee with 4 years, 11 months, 30 days is not at 5 years, and a date-difference routine can disagree on leap years.
- **Cut-off date**: service measured on the payroll run date versus the month-end date.
- **Floating-point storage**: 4.999999 stored where 5 is intended.
- **Rounding of the final payout**: applied before or after the rating factor.

2. Evaluation of the Current Test Design

Weakness	Effect
Test data chosen from "typical" employees (3, 7, 12, 20 years)	Mid-range values pass, edges never touched
No derivation of tests from the tier table	Boundaries not enumerated
Rating and service tested separately	Interaction of tier × rating untested
Exploratory style, dependent on tester intuition	Coverage differs from person to person and cannot be shown
Expected results computed by eye	Errors in the oracle, especially rounding
No traceability between tier rules and tests	Impossible to state which rules are verified

The test design was sample-based, not rule-based. A sample can only reveal a boundary defect if it happens to land on the boundary.

3. Equivalence Class Partitioning

Service years

Class	Range	Validity	Expected tier
S1	below 0 (negative)	Invalid	Rejected
S2	0 to 4	Valid	T0 (2%)
S3	5 to 9	Valid	T1 (5%)
S4	10 to 14	Valid	T2 (8%)
S5	15 and above, up to the practical maximum (say 50)	Valid	T3 (12%)
S6	above the maximum (say over 50)	Invalid	Rejected or flagged
S7	non-numeric, null, a future joining date	Invalid	Rejected

Performance rating

Class	Range	Validity	Expected factor
R1	1	Valid	0×
R2	2	Valid	0.5×
R3	3	Valid	1.0×
R4	4	Valid	1.25×
R5	5	Valid	1.5×
R6	0, negative, or above 5	Invalid	Rejected
R7	blank, non-numeric, decimal such as 3.5	Invalid	Rejected (or defined handling)

Ratings are discrete, so each of the five valid values is its own class. Treating "1 to 5" as one class would hide a wrong factor on any single rating.

4. Boundary Value Analysis

BVA gives the exact values to use in each tier. I apply the three-point approach (just below, on, just above) to every tier edge and include the outer limits.

Boundary	Below	On	Above	What it proves
Lower validity (0)	−1	0	1	New joiner handled, negatives rejected
T0/T1 (5 years)	4	**5**	6	The reported defect
T1/T2 (10 years)	9	**10**	11	The reported defect
T2/T3 (15 years)	14	**15**	16	The reported defect
Upper validity (50)	49	50	51	Maximum limit

Sub-year precision (needed because "years" is derived from dates):

Employee joined	Run date	Completed years	Expected tier
4 years, 11 months, 30 days before		4	T0
Exactly 5 years before		5	T1
5 years and 1 day before		5	T1
Leap-day joiner (29 Feb), checked on 28 Feb and 1 Mar of the fifth year		4 or 5 per policy	Confirm with HR rule

The first three rows catch the exact defect. The leap-day row protects against a second, quieter fault that exact-year integers would never show.

5. Redesigned Test Suite

Layer A: Boundary tests at the reported defect points, with a mid-rating of 3 so the factor is neutral (salary ₹50,000)

TC	Years	Rating	Expected tier	Expected bonus
B01	4	3	T0	₹1,000.00
B02	**5**	3	T1	₹2,500.00
B03	6	3	T1	₹2,500.00
B04	9	3	T1	₹2,500.00
B05	**10**	3	T2	₹4,000.00
B06	11	3	T2	₹4,000.00
B07	14	3	T2	₹4,000.00
B08	**15**	3	T3	₹6,000.00
B09	16	3	T3	₹6,000.00

B02, B05 and B08 reproduce the reported failures. B01, B04, B07 and the "above" values show the tier either side behaves correctly, so a fix that merely shifts the error is caught.

Layer B: Outer limits and invalid service values

TC	Years	Rating	Expected
B10	−1	3	Rejected
B11	0	3	T0, ₹1,000.00
B12	50	3	T3, ₹6,000.00
B13	51	3	Rejected or flagged
B14	"abc" / blank	3	Rejected

Layer C: Rating coverage on a boundary employee (years = 10, salary ₹50,000, T2 = 8% = ₹4,000 base)

TC	Rating	Factor	Expected bonus
C01	1	0	₹0.00
C02	2	0.5	₹2,000.00
C03	3	1.0	₹4,000.00
C04	4	1.25	₹5,000.00
C05	5	1.5	₹6,000.00
C06	0 / 6	n/a	Rejected
C07	3.5 / blank	n/a	Rejected

Using a boundary employee here tests rating and tier interaction at the point of highest risk.

Layer D: Tier × rating corner combinations (extremes of each tier paired with extreme ratings)

TC	Years	Rating	Salary	Expected bonus
D01	5	1	50,000	₹0.00
D02	5	5	50,000	₹3,750.00
D03	9	5	50,000	₹3,750.00
D04	10	5	50,000	₹6,000.00
D05	15	5	50,000	₹9,000.00
D06	14	5	50,000	₹6,000.00

D04 against D03 shows a jump from ₹3,750 to ₹6,000 for a single extra year, which is the intended design. Any result in between would indicate a tier mix-up.

Layer E: Money and date precision

TC	Scenario	Expected
E01	Salary ₹33,333.33, years 5, rating 4	33,333.33 × 5% × 1.25 = 2,083.33 (rounded to 2 decimals; confirm the rounding rule)
E02	Salary 0	Bonus ₹0.00
E03	Employee joined exactly 5 years before the payroll date vs 1 day short	T1 vs T0
E04	Leap-day joiner	Per HR policy
E05	Same inputs run twice	Identical output (determinism)

Total: 9 + 5 + 7 + 6 + 5 = 32 tests, each traceable to a class or boundary.

6. Why BVA and ECP Beat Exploratory Testing for This Logic

Criterion	Exploratory testing	ECP + BVA
Finding boundary defects	Depends on a tester thinking to try exactly 5, 10, 15	Boundaries are derived mechanically from the tier table, so none can be forgotten
Probability of the reported defect surviving	Real: the typical values 3, 7, 12 all pass	Near zero, because B02, B05 and B08 are required cases
Repeatability	Low; two testers produce different sessions	Complete: same inputs, same expected outputs every run
Proof of coverage	Impossible to demonstrate	Each class and boundary is a checklist item
Expected result accuracy	Calculated informally	Calculated in advance from the formula, so the oracle is checked
Regression value	Weak; cannot be re-run exactly	Easily automated and run on every build
Audit and compliance	Little evidence	Documented, traceable evidence, which payroll audits demand
Cost of a miss	Employees paid wrongly, payroll corrections, legal exposure	Defect is caught before the first payroll run

Why the financial nature of the logic matters:

- **The logic is a deterministic rule table.** Rules with sharp thresholds produce failures exactly at those thresholds. The input domain is structured, so a technique that exploits the structure is more efficient than free exploration.
- **Errors are repeated at scale.** A tier fault does not hit one transaction. It hits every employee at that service level in every payroll cycle, and underpayment or overpayment is a compliance problem.

- **Correctness is binary and measurable.** A bonus either equals the formula result or it does not. Exploration thrives where requirements are vague and judgment is needed. Here there is a precise oracle, so systematic checking is stronger.
- **Exploratory sessions are also open to anchoring.** Testers drift toward common values such as 3 or 10 years of service and may test 10 as a "round number" without realizing it is a boundary, then fail to test 9 and 11 alongside it.

Where exploratory testing still has value: discovering requirements nobody wrote down (for example, how part-year service, unpaid leave, or a mid-month promotion should affect tier). It works best as a supplement after the BVA/ECP suite, and any finding should be converted into a formal case.

7. Expected Outcome

Measure	Before	After
Tier boundaries tested	None deliberately	3 of 3, with below, on and above values
Rating classes tested	Partial	5 of 5 valid, plus invalid classes
Tier × rating corners	None	6 explicit cases
Date-derived service edge cases	Not considered	Covered (E03, E04)
Reproducibility	Low	Full
Defect recurrence risk	High	Locked out by automated regression

8. Conclusion

The payout errors at 5, 10 and 15 years are classic boundary defects, almost certainly a > versus >= mistake in the tier selection, compounded by a possible ambiguity in how years of service are derived from dates. The existing tests missed them because they sampled typical values. Equivalence class partitioning splits service years and ratings into complete valid and invalid classes, and boundary value analysis places tests at 4/5/6, 9/10/11 and 14/15/16 plus the outer limits and date edge cases. The resulting 32-case suite is small, traceable, repeatable and automatable. For rule-driven financial logic with a precise oracle, this beats exploratory testing, which is best kept as a supplementary activity for discovering undocumented rules.

7.An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.

Lab Temperature Controller: CFG, Coverage Gaps and an MC/DC Strategy

1. Failure Analysis

The alarm failed during one specific sensor-malfunction sequence. The fault is not in one decision. It sits in how the module's state (faultCount, lastValidT) behaves across successive calls.
I use this conceptual module (setpoint 37 °C, deadband ±2, alarm limits 32 °C and 42 °C):

```
1   read T, sensorOK, sensorStale, coolerFault, heaterFault
2   if (!sensorOK || sensorStale)            // D1  (A || B)
3       faultCount++
4       if (faultCount >= 3)                 // D2
5           raiseAlarm(); enterSafeState(); return
6       T = lastValidT                       // substitute last good reading
7   else
```

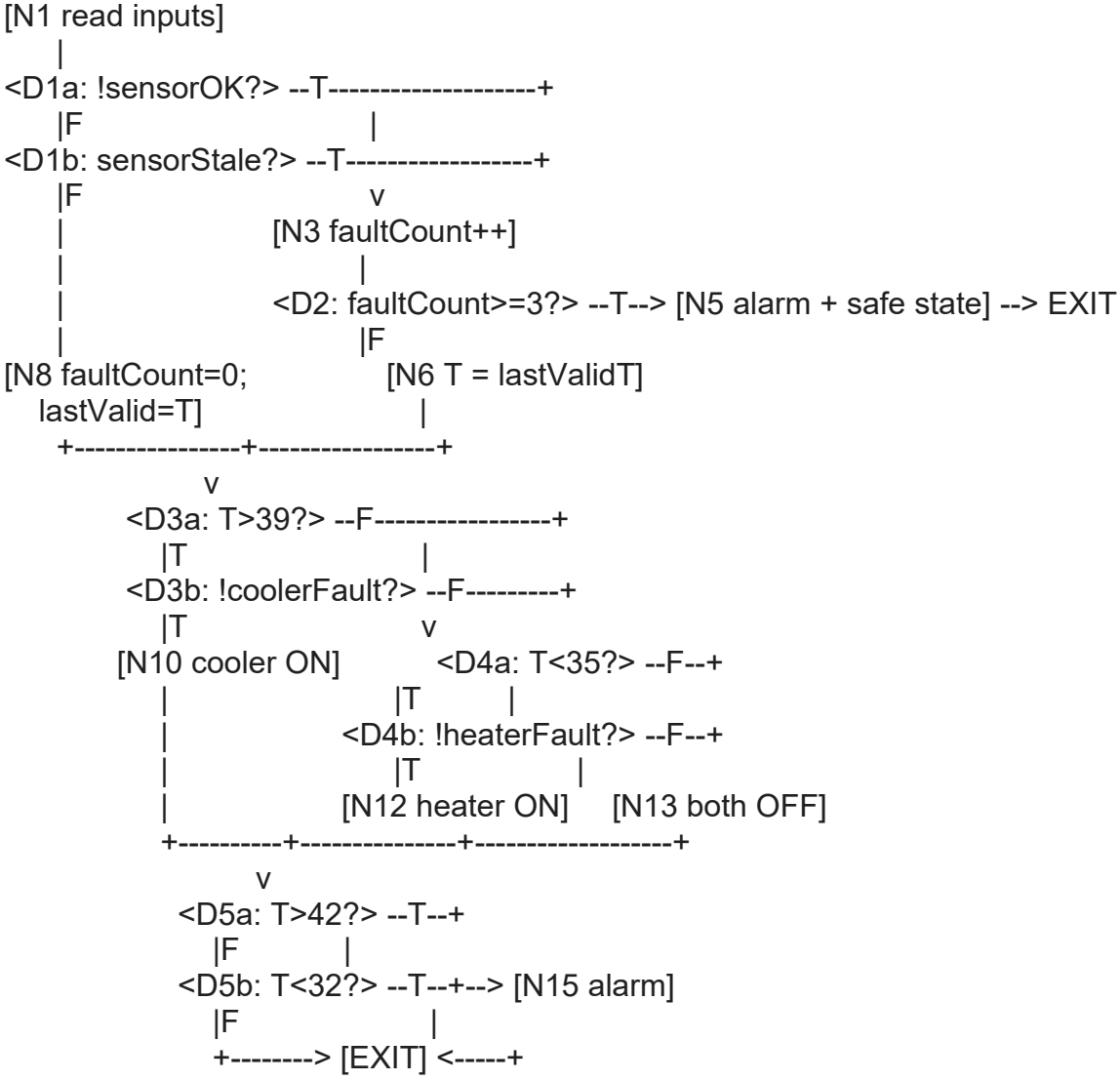
```
8       faultCount = 0; lastValidT = T        // any good reading resets the counter
9   if (T > 39 && !coolerFault)              // D3  (C && D)
10      cooler ON; heater OFF
11  else if (T < 35 && !heaterFault)          // D4  (E && F)
12      heater ON; cooler OFF
13  else both OFF
14  if (T > 42 || T < 32)                    // D5  (G || H)
15      raiseAlarm()
```

The escape sequence: fault, fault, valid, fault, fault, and so on.

Step	Sensor	faultCount	T used	Alarm?
1	Fault	1	stale 37	No
2	Fault	2	stale 37	No
3	Valid (flicker)	**reset to 0**	real	No
4	Fault	1	stale 37	No
5	Fault	2	stale 37	No

The alarm needs three *consecutive* faults, and an intermittent sensor never delivers them. Meanwhile D5 judges the **substituted** temperature, so a real excursion to 45 °C is hidden behind the frozen 37 °C. Both safeguards are blinded at once.

2. Control Flow Graph



Compound decisions are split into one node per condition, because short-circuit evaluation makes each condition a separate branch point in compiled code.

Cyclomatic complexity: the simple predicates are A, B, D2, C, D, E, F, G, H = 9, so **V(G) = 9 + 1 = 10**. At least ten independent paths exist, and a suite with fewer than ten tests cannot be adequate.

3. Evaluation of Coverage Gaps

Assume the team's suite was four tests: normal operation, over-temperature, under-temperature, and three consecutive sensor faults.

Criterion	Result	Comment
Statement	≈ 100%	Every line ran
Branch (decision outcomes)	≈ 100%	D2 is true on the third fault and false on the first two
MC/DC	**6 of 9 conditions ≈ 67%**	Independence shown for A, D2, C, E, G, H only

The suite achieved full branch coverage and still shipped the defect. This is the central finding, and there are three separate reasons.

Gap	Why branch coverage hides it	Safety consequence
B (sensorStale) never true alone	D1 became true through !sensorOK, which is enough for branch coverage	A stale-but-"OK" sensor is never exercised
D (!coolerFault) never false	D3 was false whenever T was ordinary, so the fault sub-condition was never decisive	An over-temperature with a failed cooler is untested
F (!heaterFault) never false	The same reasoning on the heating side	A dead heater is invisible
Sequence across calls	Coverage is computed per execution path, not per sequence of calls	The reset-on-valid defect lives only in the sequence
Masking by substitution	D5 tests T after line 6 rewrote it	Real temperature is never compared with limits during a fault

A further finding appears when test T6 below runs: with T = 40 and a failed cooler, the module does nothing and raises no alarm, because 40 is below the 42 limit. A failed actuator during an out-of-band temperature is silently tolerated. This is a requirements gap, not only a test gap, and it should go back to the safety analysis.

4. White-Box Strategy for 100% MC/DC

MC/DC requires that every condition be shown to independently change its decision's outcome while the other conditions are held fixed. For *n* conditions this needs a minimum of *n* + 1 tests, against 2^n for exhaustive combinations.

Step 1: Independence pairs per decision

Decision	Truth function	Test rows (A,B / C,D / E,F / G,H)	Independence pairs
D1	A ∨ B	(F,F)→F, (T,F)→T, (F,T)→T	A: rows 1,2. B: rows 1,3
D3	C ∧ D	(T,T)→T, (T,F)→F, (F,T)→F	C: rows 1,3. D: rows 1,2
D4	E ∧ F	(T,T)→T, (T,F)→F, (F,T)→F	E: rows 1,3. F: rows 1,2
D5	G ∨ H	(F,F)→F, (T,F)→T, (F,T)→T	G: rows 1,2. H: rows 1,3
D2	faultCount ≥ 3	2 → F, 3 → T	boundary pair

Step 2: Merged executable test set (10 tests, matching V(G))
D4 is reachable only when D3 is false, and D5 follows every non-returning path, so the rows are merged into shared executions. Initial state: faultCount = 0, lastValidT = 37.

TC	Inputs	A	B	D2	C	D	E	F	G	H	Result
T1	OK, fresh, T = 37	F	F	–	F	T	F	T	F	F	Idle, no alarm (shared base)
T2	Sensor not OK, count 0	**T**	F	F	F	T	F	T	F	F	Substitute 37, idle
T3	OK but stale, count 0	F	**T**	F	F	T	F	T	F	F	Substitute 37, idle
T4	Sensor not OK, count 2	T	F	**T**	–	–	–	–	–	–	Alarm + safe state, return
T5	OK, T = 40, cooler OK	F	F	–	T	**T**	F	T	F	F	Cooler ON
T6	OK, T = 40, cooler failed	F	F	–	T	**F**	F	T	F	F	Neither actuator, no alarm
T7	OK, T = 34, heater OK	F	F	–	F	T	T	**T**	F	F	Heater ON
T8	OK, T = 34, heater failed	F	F	–	F	T	T	**F**	F	F	Neither actuator
T9	OK, T = 43	F	F	–	T	T	F	T	**T**	F	Cooler ON + alarm
T10	OK, T = 31	F	F	–	F	T	T	T	F	**T**	Heater ON + alarm

Independence achieved:

Condition	Pair	Condition	Pair
A	T1, T2	E	T1, T7
B	T1, T3	F	T7, T8
D2	T2, T4	G	T1, T9
C	T1, T5	H	T1, T10
D	T5, T6		

All 9 conditions are covered, so MC/DC is **100%**. Each pair differs in exactly one condition and flips that decision's outcome. Where short-circuit evaluation skips a condition, the pair uses the nearest test that actually evaluates it.

Step 3: Sequence tests (MC/DC cannot reach these)
MC/DC is a per-execution criterion. The reported failure needs **stateful test sequences**, so I add them as a separate mandatory layer.

TC	Sequence	Real temperature	Required behavior
S1	fault, fault, valid, fault, fault	drifts 37 → 45	Alarm must fire (windowed or cumulative fault count)
S2	fault ×1 held for N cycles	rises past 42	Alarm on timeout, not on substituted data
S3	fault, fault, fault (exact boundary)	stable	Alarm exactly on the third, not the second or fourth
S4	valid, fault, valid, fault, valid, fault	stable	Alarm per policy for repeated intermittent faults
S5	sensor fault while cooler fault is active	40	Alarm (dual-fault policy)
S6	recovery: 3 valid readings after alarm	normal	Alarm latches until reset by an operator

Step 4: Corrective design these tests demand

- Replace "consecutive" counting with a **leaky or windowed counter** (for example, 3 faults in any 10 samples).

- Never evaluate alarm limits on substituted values alone. Alarm on staleness duration.
- Make the safe state the **default** after an unresolved sensor fault.
- Treat an actuator fault plus an out-of-band temperature as an alarm condition.

Step 5: Verification of the tests themselves
Apply mutation testing on the operators and constants (|| to &&, >= 3 to > 3, remove faultCount = 0, delete the lastValidT assignment). A suite that does not kill these mutants has weak assertions. Also confirm that no path is infeasible only because of a masking dependency, and document each such case.

5. Impact on Safety-Critical Reliability

Aspect	Before	After
MC/DC	≈ 67% (6 of 9)	100% (9 of 9)
Branch / statement	≈ 100%	100% (retained)
Actuator-fault conditions (D, F)	Never decisive	Shown to control the outcome
Stale-sensor condition (B)	Never isolated	Isolated and verified
Intermittent-fault sequences	None	S1 to S6
Masked-alarm defect	Shipped	Caught by T3 and S1/S2
Test count	Ad hoc, 4	10 MC/DC + 6 sequence

Why this raises reliability:

- **It proves each condition matters.** In a safety interlock, a condition that never changes the outcome in testing is a condition whose failure you have not observed. MC/DC removes that blind spot, which branch coverage leaves open.
- **It scales.** The test count grows linearly with conditions (n + 1 per decision), so it stays affordable for dense if-else logic, where exhaustive combinations would explode.
- **It targets the dangerous operators.** The most common safety logic faults are && versus || and a dropped negation. MC/DC detects exactly those, since a wrong operator changes which pairs flip the outcome.
- **It is recognized practice.** Avionics software (DO-178C Level A) mandates MC/DC, and functional-safety standards such as IEC 61508 recommend it for the highest integrity levels. Using it on a lab controller supplies auditable evidence for regulated or high-value experiments.
- **It is honest about its limit.** MC/DC passed would not have prevented this release defect alone. The reset-on-valid fault needed the sequence layer and a safety-requirements review. Reliability comes from combining the two.

6. Conclusion
The controller shipped with 100% statement and branch coverage and still failed, because the failure combined an unexercised condition set (B, D, F), a state counter that resets on any valid reading, and an alarm check that tests substituted data. The CFG shows V(G) = 10 independent paths and nine atomic conditions. Raising the suite to 10 merged MC/DC tests proves every condition independently drives its decision, and six sequence tests expose the intermittent-fault escape that no single-call criterion can. Together with the corrective design (windowed fault counting, alarming on staleness, a fail-safe default), this gives auditable, safety-grade assurance that the alarm will fire when the sensor misbehaves.

8.A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.

Negative Totals in a Travel Booking Portal: Combination Testing with Decision Tables and Cause-Effect Graphing

1. Analysis of the Failure

A negative total means the discounts together exceed the price. Each promo is acceptable alone, but their sum (or their order of application) is not. This is a **combination defect**: it depends on price, which promos are stacked, and the user's tier at the same time, so it cannot show up when one field is tested alone.

Assumed specification (none is given, so I fix one to make the design concrete):

Item	Rule
Base price	₹500 to ₹5,00,000
User tier	Standard (S), Silver (Sv), Gold (G)
Promo types	**P** = percentage (20% off), **F** = flat (₹1,500 off), **L** = loyalty (extra 10% off, Silver and Gold only)
Stacking	Maximum 3 codes, one per type; codes apply in the order P, then F, then L
Floor rule	Total must never fall below ₹0. Business policy: a minimum payable of ₹100 (a convenience fee)
Tier cap	Combined discount capped at 20% (S), 35% (Sv), 50% (G)

2. Limitations of Testing Inputs in Isolation

Limitation	Example from this portal
Each promo passes alone	₹1,500 flat off a ₹2,000 fare is fine, ₹1,500 off after a 20% discount on ₹1,800 is not (1,800 − 360 − 1,500 = −60)
Price is not varied alongside promos	A flat discount is harmless on a ₹50,000 fare and fatal on a ₹1,200 fare
Tier eligibility never crossed with promo type	The loyalty code rejected for Standard users is never tried in a stack
Order of application not tested	Flat-then-percent gives a different total from percent-then-flat
Floor logic tested only at the end	Intermediate negatives, rounding drift, and a clamp applied too late go unseen
Single-promo tests give false confidence	100% of "each code works" tests pass while the defect sits in stacking

Size of the space: price classes × 8 promo subsets × 3 tiers × 2 orders. Even with 6 price classes that is 6 × 8 × 3 × 2 = 288 combinations, and with real values it is unbounded. Systematic reduction is needed.

3. Cause-Effect Graph

Causes (inputs, each true or false):

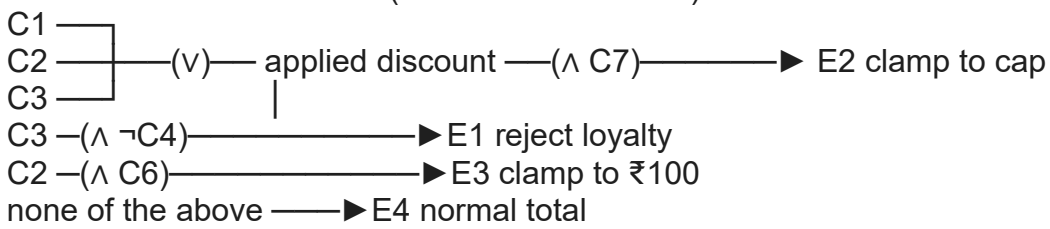
ID	Cause
C1	Percentage promo applied
C2	Flat promo applied
C3	Loyalty promo applied
C4	Tier is Silver or Gold
C5	Tier is Gold
C6	Price after percentage discount is at most the flat amount (price ≤ 1,875 when P applied; ≤ 1,500 otherwise) — "small price"
C7	Combined discount exceeds the tier cap

Effects:

ID	Effect
E1	Loyalty code rejected (ineligible tier)
E2	Discount clamped to the tier cap
E3	Total clamped to the minimum payable (₹100)
E4	Total computed normally
E5	Invalid input message (price out of range, more than 3 codes, duplicate type)

Logic (intermediate nodes):

C3 ∧ ¬C4 → E1 (loyalty needs Silver/Gold)
(C1 ∨ C2 ∨ C3 valid) ∧ C7 → E2 (cap reached)
(C2 ∧ C6) → E3 (flat exceeds what is left)
¬E1-trigger ∧ ¬C7 ∧ ¬C6 → E4
C5 modifies C7's threshold (50% vs 35% vs 20%)



Constraints encoded in the graph:

- C5 implies C4 (Gold is also Silver-or-above).
- E1 masks C3 (a rejected code contributes nothing to the discount).
- C6 is only evaluable once price is known, so price and promos are linked at the graph level, not at input level.

4. Decision Table

Conditions: **Price class** (Low ≤ 1,500, Mid 1,501 to 5,000, High above 5,000), **Promos applied**, **Tier**.

Rule	Price	Promos	Tier	Loyalty eligible	Cap breached	Floor breached	Expected action
R1	Any	none	Any	–	N	N	Total = price
R2	Mid/High	P	Any	–	N	N	20% off
R3	Mid/High	F	Any	–	N	N	₹1,500 off
R4	Mid/High	L	S	N	–	–	Reject loyalty, total = price
R5	Mid/High	L	Sv/G	Y	N	N	10% off

R6	Mid	P+F	S	–	**Y** (cap 20%)	N	Discount clamped to 20%
R7	High	P+F	S	–	**Y**	N	Clamped to 20%
R8	High	P+F	Sv	–	N (about 20% + small)	N	Both applied
R9	Low	F	Any	–	**Y**	**Y**	Clamp to ₹100 (flat exceeds price)
R10	Low	P+F	Any	–	Y	**Y**	Clamp to ₹100, never negative
R11	Mid	P+F+L	Sv	Y	**Y** (cap 35%)	N	Clamped to 35%
R12	Mid	P+F+L	G	Y	N or Y	N	Clamped to 50% if exceeded
R13	Low	P+F+L	G	Y	Y	**Y**	Clamp to ₹100
R14	Any	P+P (duplicate)	Any	–	–	–	Reject, duplicate type
R15	Any	4 codes	Any	–	–	–	Reject, over limit
R16	Out of range	Any	Any	–	–	–	Reject, invalid price
R17	Any	P+F+L	S	**N**	Y/N	N	Loyalty rejected, remaining P+F capped at 20%

Infeasible combinations removed (no test needed):

Combination	Reason
Loyalty applied with Standard tier	Contradicts E1
Floor breached with no flat or no stacking at price ≥ 2,000	The maximum discount (50%) cannot reach the floor at that price
Cap breached with a single percentage promo for Standard	20% = the cap, not above it

Eliminating them reduces 288 candidate combinations to the 17 rules above.

5. Optimized Test Set

Each rule gets one test, boundary values are added for the two thresholds that decide the outcome (floor and cap), and rules that cannot differ in outcome are merged.

Core tests from the decision table (with computed expected totals):

TC	Price (₹)	Promos	Tier	Expected total	Rule
T01	3,000	none	S	3,000.00	R1
T02	3,000	P	S	2,400.00	R2
T03	3,000	F	S	1,500.00	R3
T04	3,000	L	S	3,000.00 (rejected)	R4
T05	3,000	L	Sv	2,700.00	R5
T06	3,000	P+F	S	1,500 stacked would be 900.00; cap 20% gives 2,400.00	R6
T07	10,000	P+F	Sv	10,000 − 2,000 − 1,500 = 6,500 (35% of 10,000 = 3,500, so 3,500 is allowed): 6,500.00	R8
T08	1,200	F	G	Clamp to 100.00 (never −300)	R9
T09	1,800	P+F	G	1,800 − 360 − 1,500 = −60, clamp to 100.00	R10

T10	3,000	P+F+L	Sv	Cap 35%: 1,950.00	R11
T11	3,000	P+F+L	G	20% + 1,500 + 10% = 2,400 + ... hits 50% cap: 1,500.00	R12
T12	1,200	P+F+L	G	Clamp to 100.00	R13
T13	3,000	P+P	Any	Reject	R14
T14	3,000	4 codes	Any	Reject	R15
T15	499 / 5,00,001	none	Any	Reject	R16
T16	3,000	P+F+L	S	Loyalty rejected, cap 20%: 2,400.00	R17

Boundary tests on the two deciding thresholds:

TC	Purpose	Price (₹)	Promos	Tier	Expected
B01	Floor exactly	1,600	F	S	100.00 (1,600 − 1,500)
B02	One rupee below floor	1,599	F	S	100.00 (clamped, 99 would fall short)
B03	One rupee above floor	1,601	F	S	101.00
B04	Cap exactly (Standard, 20%)	3,000	P	S	2,400.00 (not clamped, equal)
B05	Discount just over cap	3,000	P+F	S	2,400.00 (clamped)
B06	Silver cap edge	10,000	P+F+L	Sv	6,500.00 (exactly 35%)
B07	Price lower limit	500	F	G	100.00
B08	Price upper limit	5,00,000	P+F+L	G	2,50,000.00 (50% cap)

Order-of-application tests:

TC	Check	Expected
O01	Same codes entered in different UI order (F, P versus P, F) at price 1,800, tier G	Identical total (100.00) either way, proving order is applied by rule, not entry sequence
O02	Add a code, remove it, add another	Total recalculated from the base, never from a stale intermediate
O03	Rounding: ₹1,999.99 with P and L	Totals match to two decimals under the defined rounding rule

Total: 16 + 8 + 3 = 27 tests, replacing 288 combinations.

6. How Redundancy Is Minimized and Defects Maximized

Technique	Redundancy removed	Defect class targeted
Infeasible-combination elimination	Contradictory cases never written	Wasted effort
One test per decision rule	Combinations producing the same outcome are merged	Missed rule logic
Boundary tests only at deciding thresholds (floor, cap)	No boundaries on non-deciding values	Off-by-one at the clamp
Price classes instead of price values	Every price in a class behaves identically	Price-sensitive flat discounts
Cause-effect masking (E1)	Rejected-loyalty cases collapse into the plain case	Eligibility bugs
Order tests as a separate small set	Order is checked once, not in all 288 combinations	Sequence defects

Coverage achieved:

Measure	Result
Feasible decision rules	17 of 17
Causes C1 to C7, each true and false	Yes
Effects E1 to E5 triggered	Yes
Floor and cap boundaries	Below, at, above
Promo subsets (8)	All 8 appear
Tiers (3)	All 3 appear with each deciding effect
Test reduction	288 combinations to 27, about 90% fewer

7. Justification

- **Why decision tables with cause-effect graphs:** the defect is a rule interaction, and these methods enumerate rules directly rather than relying on a tester to stumble onto a bad stack. The graph also exposes infeasible and masked combinations early.
- **Why not pairwise alone:** the negative total needs three factors at once (low price, flat promo, a stacked percentage), so two-way coverage can pass while the failing triple is never produced. The decision table covers it explicitly.
- **Why not exhaustive:** 288 combinations with fixed values, unbounded with real values, and most share an outcome.
- **Honest limit:** the suite proves the clamps and caps work for the specified rules. It cannot reveal an unlisted promo type or a new tier. Adding one means adding a cause to the graph and its rows to the table, which keeps growth controlled.

8. Recommended Fix Tested by This Suite

Enforce the floor as a **final, unconditional clamp** (total = max(total, 100)) after all discounts, apply discounts in a fixed documented order, and reject ineligible codes before they contribute. T08, T09, T12, and the B-series fail against the current behavior and pass after the fix.

9. Conclusion

Testing promo codes, price, and tier separately cannot expose a negative total, because the failure exists only in their combination. Cause-effect graphing identified seven causes and five effects and revealed the masked and infeasible cases. The decision table reduced 288 combinations to 17 feasible rules, and boundary and ordering tests on the floor and cap thresholds brought the final set to 27 tests. That set covers every rule, every threshold edge and every tier, and would have caught the negative-total defect before release.

9.A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.

Legacy Migration Tool: Why Statement Coverage Missed the Infinite Loops

1. Failure Analysis

An infinite loop is a termination defect. Statement coverage only records that a line ran at least once. It cannot show that a loop exits after the right number of iterations, so a suite can reach 100% and still never test whether the loops end.

I use this conceptual module, with MAX = 3 repair attempts per record:

```
1   batch = 0; errors = 0
2   while (hasMore())                          // P1  outer loop
3     rows = fetch(batch)
4     if (rows.isEmpty() && !eof) continue       // P2  (P2a && P2b), skips batch++
5     for (r in rows)                          // P3  middle loop
6       tries = 0
7       while (!valid(r))                      // P4  inner loop
8         if (repairable(r))                   // P5
9           repair(r); tries++
10          if (tries > MAX) { quarantine(r); errors++; break }   // P6
11        else if (r.parentMissing)              // P7
12          refetchParent(r); continue           // no tries++
13        else { quarantine(r); errors++; break }
14      if (r.modified) write(r)                 // P8
15    batch++
16    if (errors > LIMIT) abort()                // P9
```

There are two hidden infinite loops, and both come from a continue that skips the statement that makes progress:

Location	Why it never terminates	Production trigger
Line 12	continue bypasses tries++, so the inner loop has no decreasing counter on this path	Circular foreign keys (A needs B, B needs A), or a parent that never arrives
Line 4	continue bypasses batch++, so the outer loop re-fetches the same batch	Source returns an empty page without setting eof, for example during a connection stall

2. Evaluation of the Current White-Box Approach

Assume three tests: one batch with a valid record, one repairable record, one record whose parent appears after a single refetch, and one unrepairable record, followed by an empty batch that is followed by data.

Observation	Why it hid the defect
Line 12 executed once	The mocked parent always appears on the first refetch, so the loop exits. Statement coverage marks the line as covered.
Line 4's continue executed once	The mock returns data on the second fetch. The spin never occurs.
Test doubles always cooperate	valid(), eof and refetchParent() behave ideally, so no condition that depends on external state is ever adversarial.
Coverage tool counts hits, not repetitions	One execution and one million executions look identical.
No loop-variant reasoning	Nobody asked what value strictly decreases on every iteration of each loop.

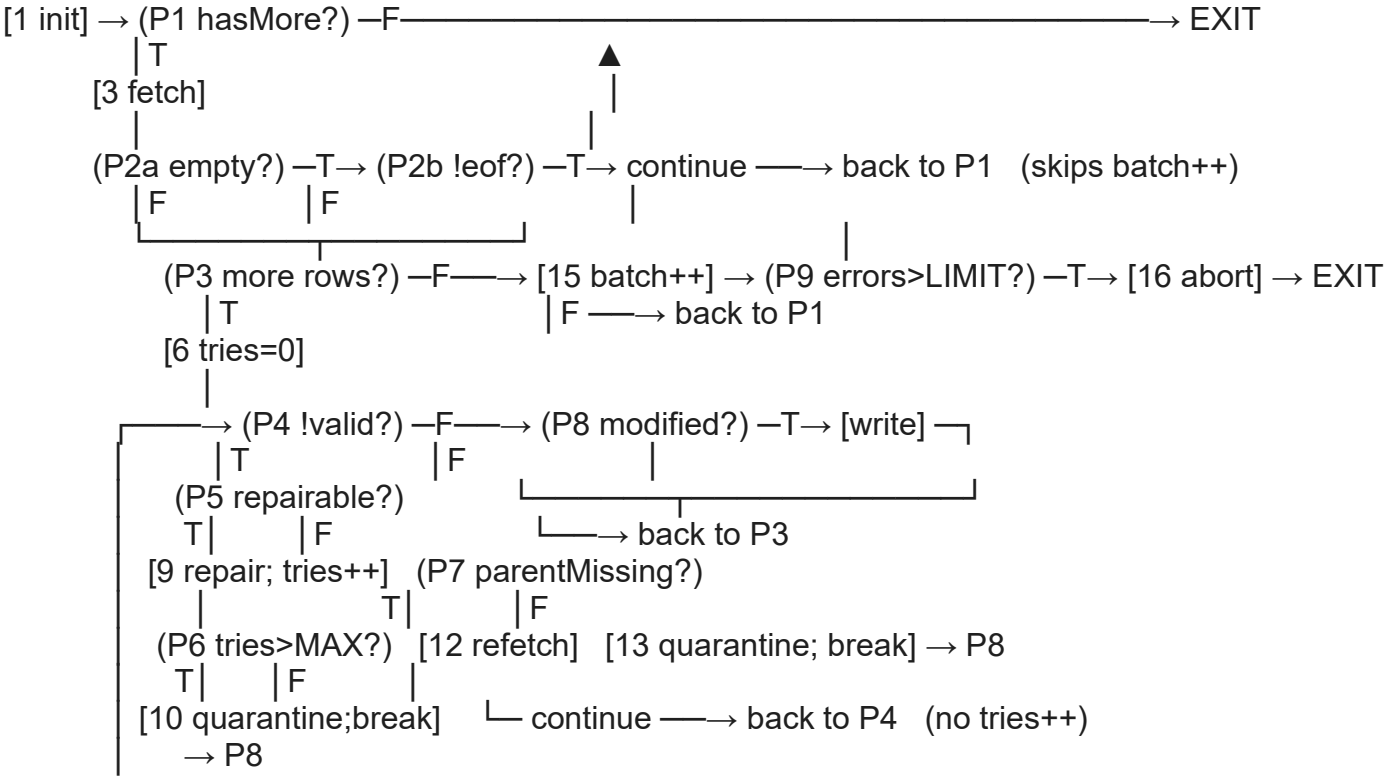
Why statement coverage is insufficient here:

1. **It measures execution, not repetition.** An infinite loop needs the same statements to execute without bound.
2. **It ignores decision outcomes.** Outcomes with no statement of their own (for example, an empty else such as P2b = false, or P6 = false) can stay untested.
3. **It cannot see paths.** The defect is a path through lines 7, 8 or 11, then 12 and back to 7, taken repeatedly.
4. **It cannot check state progress.** Whether tries and batch change on every route is a data-flow question.

Assumed measured results of the existing suite:

Criterion	Achieved	Comment
Statement	100%	Reported to management
Branch	about 90% (18 of 20 outcomes)	P2b = F and P6 = F untested
Basis paths	about 36% (4 of 11)	Unmeasured
Loop boundary cases	2 of 17	Only 0 and 1 iterations tried
Termination checks	none	The real gap

3. Control Flow Graph and Cyclomatic Complexity



└──────────────────────── F edge of P6 and edge from [12]

All compound conditions are split, because short-circuit evaluation makes each simple condition a separate branch point in compiled code.

Simple predicates: P1, P2a, P2b, P3, P4, P5, P6, P7, P8, P9, which is 10.
V(G) = 10 + 1 = 11, so at least **11 independent paths** must be exercised.

Interpretation	Meaning here
11 is above the commonly cited threshold of 10	Beyond the comfortable single-function range
break and continue are unstructured exits	Essential complexity is greater than 1, so the code cannot be reduced to clean structured blocks without refactoring
Three nesting levels	Paths multiply across levels

Total path count: assume about 6 distinct outcomes per record, 3 records per batch and 2 batches. That gives roughly $6^6 \approx$ **4.7×10^4 paths**, and the figure grows without bound as data size grows. Exhaustive path testing is impossible, which is why V(G) is the right yardstick.

4. Path Testing Strategy

Layer 1: Basis path set (11 tests, one valid basis)

BP	Scenario	Decisions exercised
BP1	Zero batches	P1 = F immediately
BP2	One batch, one valid unmodified record (baseline)	P2a F, P3 T, P4 F, P8 F, P9 F
BP3	Empty batch, not at eof, then data	P2a T, P2b T, continue edge
BP4	Empty batch at eof	P2a T, P2b F, P3 skipped
BP5	Valid record that is modified	P8 T
BP6	Invalid, repairable, fixed on first repair	P4 T, P5 T, P6 F
BP7	Repairable but never valid, exceeds MAX	P6 T, break
BP8	Invalid, parent missing, refetch then valid	P7 T, continue edge
BP9	Invalid, unrepairable, parent present	P7 F, quarantine and break
BP10	Errors exceed LIMIT	P9 T, abort
BP11	Two or more batches	P1 back-edge taken

BP3 and BP8 are the two paths that contain the production defects. Selecting them is mandatory under basis path testing, which is a decisive advantage over statement-driven selection.

Layer 2: Loop boundary conditions (nested loop method)

For a simple loop with maximum *n*, test 0, 1, 2, a typical value, *n* − 1, *n* and *n* + 1. For nested loops, test the innermost loop first with the outer loops at their minimum, then move outward holding the inner loops at typical values, and finally combine extremes.

Loop	Values	New tests beyond Layer 1
Inner (tries, MAX = 3)	0, 1, 2, 3, 4	tries = 2; tries = 3 then valid (2 tests)
Middle (rows per batch, size N)	0, 1, 2, typical, N − 1, N, N + 1	2 rows; typical; N − 1; N; N + 1 (5 tests)
Outer (batches)	0, 1, 2, many	many batches (1 test)
Error limit	LIMIT − 1, LIMIT, LIMIT + 1	errors = LIMIT with no abort (1 test)
Combined extremes	Inner at MAX, middle at 2, outer at 2	1 stress test

10 new tests. Exhaustive combination of seven values per loop would need $7^3 = 343$ tests, so the nested method cuts that by about 97%.

Layer 3: Termination tests (the layer statement coverage lacks)

Each loop gets a **variant**, a quantity that must strictly decrease on every iteration, and an adversarial environment that tries to prevent progress.

TT	Adversarial condition	Variant it checks	Expected behavior
TT1	Circular foreign keys (A needs B, B needs A)	refetch count	Terminates by quarantine within a bounded number of refetches
TT2	Parent never arrives	refetch count	Same
TT3	Source returns empty and never sets eof	stalled fetch count	Fails the batch after a timeout or retry cap
TT4	repair() is a no-op that leaves the record invalid	MAX − tries	Ends at tries = 4 with quarantine (confirms tries++ is the effective guard)
TT5	Exception in a callee skips batch++	batch	Batch counter still advances, or the run aborts

Harness requirement: wrap each loop with an iteration counter in test builds, and fail the test if any loop exceeds a documented bound. Without this, an infinite loop appears as a hung test run, not as a clean failure.

Layer 4: Data flow testing

Cover every definition-use pair of tries, batch and eof. The definition of tries on line 9 must reach its use on line 10 on every path through the inner loop. Line 12 provides no definition, and the data-flow check flags it directly. The same applies to batch on the line-4 path.

Layer 5: Mutation testing

Apply these mutants: delete tries++; move batch++ before the continue; change tries > MAX to >=; remove the break; change && to || in P2. Require a kill rate of at least 85%. Any surviving mutant shows assertions that cannot tell a terminating loop from a spinning one.

5. Justification Using Cyclomatic Complexity

Criterion	Tests needed here	Detects the infinite loops?
Statement	3	No, since line 12 and line 4's continue already ran once
Branch	at most V(G) = 11	Not reliably, because each outcome is taken once and not repeated
Basis path	**V(G) = 11**	**Yes**, because BP3 and BP8 are mandatory routes
Full path	about 4.7×10^4 and unbounded	In theory yes, but infeasible
Basis path + loop + termination	**26**	**Yes**, with bounded loops proven by the variant tests

Three properties of V(G) support the decision:

1. It is a lower bound for independent-path adequacy. Any suite with fewer than 11 tests provably misses an independent route. The team's four tests could not have been enough.
2. It is an upper bound for branch coverage tests. Branch coverage needs at most V(G) tests, so the cost of moving past statement coverage is known and small.
3. It does not grow with iteration count. V(G) stays at 11 however many loops iterate, which makes the basis set finite where full path coverage is not. Loop boundary and termination tests then add the iteration dimension that V(G) cannot express.

Refactoring recommendation from the metric: extract the inner logic into validateRecord(r) with predicates P4 to P7, giving V = 5, and keep processBatches() with P1, P2a, P2b, P3, P8, P9, giving V = 7. Both are below 10, and the loop variants become explicit and checkable. The total of 12 is one more than the original 11 because the extracted call adds an entry point. This is a normal cost of decomposition.

6. Expected Outcomes

Measure	Before	After
Statement coverage	100% (misleading)	100%
Branch coverage	about 90%	100%
Basis path coverage	4 of 11	11 of 11
Loop boundary cases	2 of 17	17 of 17
Termination checks	none	5 adversarial tests plus per-loop iteration bounds
Line-12 and line-4 spins	shipped	caught by BP3, BP8, TT1 to TT3
Suite size	3	26 (11 basis + 10 loop + 5 termination)
Effectiveness evidence	none	mutation score of at least 85%

7. Conclusion

Statement coverage was insufficient because an infinite loop is a property of repetition and progress, and a coverage counter records neither. The two continue statements skipped the only instructions that advanced the loop state, and the cooperative test mocks never forced the loops to repeat. Computing V(G) = 11 from the CFG gives a finite, justified set of independent paths that necessarily includes both defective routes. Nested-loop boundary tests add the iteration dimension, and termination tests with adversarial data and per-loop variants show the loops actually end. Refactoring to keep each function's V(G) at or below 10 keeps those guarantees maintainable in the legacy code.

10.A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.

Real-time Tracker Stuck on Driver Cancellation: State Transition Redesign

1. Failure Analysis

The tracker is a **state machine**. Its behavior depends on the current state and the event that arrives, not on the event alone. The reported bug is the event "driver cancels" arriving in *Out for Delivery*, where the original design has no defined next state. The app waits for an update that nothing will send.

Original model (as the team likely understood it):

Ordered → Preparing → Out for Delivery → Delivered

The gap: *Out for Delivery* has one exit (delivery completed). A driver cancellation there is an unmodeled event. The app has no "find another driver" state, no timeout, and no failure path, so it becomes a trap state with a spinner and no way out.

2. Evaluation of the Current Black-Box Strategy

Weakness	Effect
Tests follow the happy path (Ordered, Preparing, Out, Delivered)	Only 1 of the possible routes is exercised
"Driver cancels" tested as a single event, probably before pickup	The mid-delivery case is never reached
ECP and BVA applied to individual screens or inputs	Sequence and history are invisible to them
No state model, no transition table	Nobody can see which state-event pairs are unspecified
Time-based behavior (no driver found, stalled updates) untested	Timeouts, the usual exit from waiting states, are missing
Real-world dynamics (lost connection, app relaunch, out-of-order push) ignored	Stale or inconsistent UI states
The requirement itself was incomplete	Testing from the spec alone could never expose a transition the spec omits

The last point matters most. The defect is a **specification gap**, and only a model-based method surfaces gaps in the spec.

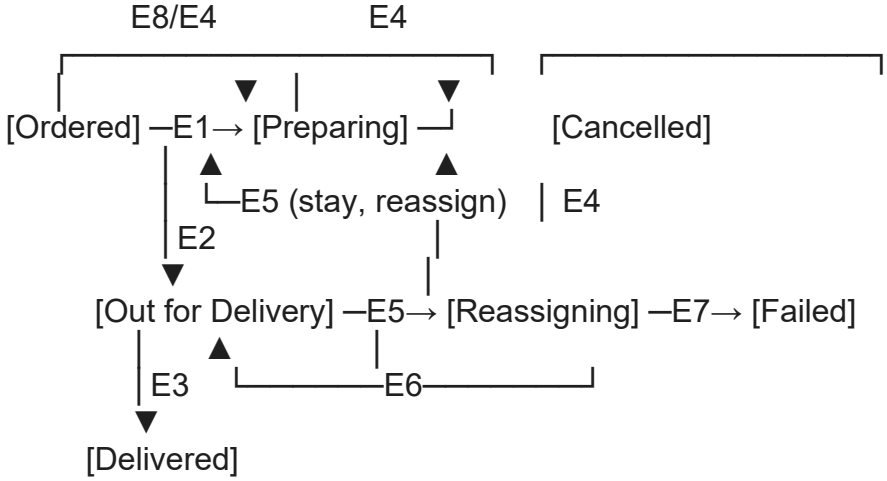
3. Corrected State Model

I add two states the original design lacked and assume these rules:

- **Reassigning:** food is picked up, the driver is gone, and the app searches for a new driver.
- **Failed:** reassignment gave up, so the order is refunded and support is notified.
- A maximum of 2 reassignments and a 10-minute search timeout (assumed for the design).

States: S1 Ordered, S2 Preparing, S3 Out for Delivery, S4 Delivered, S5 Cancelled, S6 Reassigning, S7 Failed.

Events: E1 restaurant accepts, E2 driver picks up, E3 driver marks delivered, E4 user cancels, E5 driver cancels, E6 new driver assigned, E7 search timeout, E8 restaurant rejects.



4. Transition Table

Rows are states, columns are events. A dash means the event is **invalid** in that state. The expected response is "ignore, keep state, no UI change or error."

State	E1	E2	E3	E4	E5	E6	E7	E8
Ordered	Preparing	–	–	Cancelled	–	–	–	Cancelled
Preparing	–	Out	–	Cancelled	Preparing (driver swapped)	–	–	–
Out for Delivery	–	–	Delivered	– (cancel refused)	**Reassigning**	–	–	–
Reassigning	–	–	–	Cancelled (refund)	–	Out	Failed	–
Delivered	–	–	–	–	–	–	–	–
Cancelled	–	–	–	–	–	–	–	–
Failed	–	–	–	–	–	–	–	–

Counts: 7 states × 8 events = 56 cells. There are 11 valid transitions and 45 invalid cells.

5. Redesigned Test Scenarios

Layer A: Valid transition coverage (0-switch), 8 paths cover all 11 transitions

TC	Path	Transitions covered	Expected final UI
A1	Ordered →E1 Preparing →E2 Out →E3 Delivered	E1, E2, E3	"Delivered", rating prompt, tracker closed
A2	Ordered →E4 Cancelled	E4 (Ordered)	"Cancelled", refund shown
A3	Ordered →E8 Cancelled	E8	"Restaurant declined", refund
A4	Ordered → Preparing →E4 Cancelled	E4 (Preparing)	Cancelled, fee per policy
A5	Preparing →E5 Preparing →E2 Out →E3 Delivered	E5 (Preparing)	Driver name changes, order continues
A6	Out →E5 Reassigning →E6 Out →E3 Delivered	E5 (Out), E6	"Finding a new driver", then resumes with new driver and ETA
A7	Out →E5 Reassigning →E7 Failed	E7	"Delivery failed", refund, support link
A8	Out →E5 Reassigning →E4 Cancelled	E4 (Reassigning)	Cancelled, refund

A6 is the reported bug. The assertion goes beyond "no crash" and covers the status text, the map and ETA, the available buttons, the backend order state, and the payment state, which must all agree.

Layer B: Sequence and loop coverage (1-switch), 3 tests

TC	Sequence	Purpose
B1	Out →E5 Reassigning →E6 Out →E5 Reassigning →E6 Out →E3	Two reassignments, the permitted maximum
B2	Out →E5 →E6 →E5 →E6 →E5	Third cancellation: the cap is reached and the order goes to **Failed**
B3	Preparing →E5 (driver swapped) →E2 →E5	Same event, different state, different outcome

The B tests apply BVA inside the model. The reassignment counter is tested at 0, 1, 2 (the maximum) and 3 (over the limit). The timeout is tested at 9:59, 10:00 and 10:01, which shows that the two techniques complement each other.

Layer C: Invalid transitions (the 45 "–" cells), 33 tests

Group	Cells	Tests
Non-terminal states (Ordered 5, Preparing 5, Out 6, Reassigning 5)	21	All 21
Terminal states (Delivered, Cancelled, Failed) × high-risk events E3, E4, E5, E6	12	12
Remaining terminal cells	12	Skipped, same code path as the tested ones

Expected result in every case: state unchanged, no UI change, no crash, no duplicate payment or refund. Examples:

- A late "delivered" signal after Cancelled must not mark the order delivered.
- "New driver assigned" while the app is in Out for Delivery must not duplicate the tracker.
- User cancel in Out for Delivery must show a refusal message and leave the order intact.

All 33 are one parameterized, data-driven test over the transition table.

Layer D: Dynamic and environmental scenarios, 6 tests

TC	Scenario	Expected
D1	Connection lost while in Reassigning, then restored	State resynced from the server, no stale spinner
D2	App killed in Reassigning, relaunched	Correct state restored from the server
D3	Push "Delivered" arrives before "Out for Delivery"	Out-of-order message handled, or ignored with resync
D4	Duplicate E5 notification	Treated once, the second one is ignored
D5	User cancel and driver cancel at the same instant	One deterministic outcome, one refund
D6	No update received for N minutes in any waiting state	Staleness timeout shows a refresh or support option

Total: 8 + 3 + 33 + 6 = 50 tests. The transition table and the full 56 cells are covered by the first three layers.

6. Coverage Achieved

Criterion	Target	Result
All states visited	7 of 7	Achieved (A1 to A8)
All valid transitions (0-switch)	11 of 11	Achieved
Invalid cells with defined response	45 of 45 (33 tested, 12 equivalent)	Achieved
Loops (Out ↔ Reassigning)	0, 1, 2, 3 repetitions	Achieved (B1, B2)
Trap-state check	Every non-terminal state has at least one exit	Verified by table review
Timeout boundaries	9:59, 10:00, 10:01	Achieved

7. Why State Transition Testing Fills the Gaps ECP and BVA Leave

Gap	Why ECP/BVA miss it	How state transition testing handles it
Same input, different outcome	ECP treats "driver cancels" as one class, so one test appears sufficient. The result depends on the state: swap in Preparing, reassign in Out for Delivery.	The table crosses every event with every state, so the same event is tested in each
History dependence	ECP and BVA treat each test as independent of what came before	Test cases are *sequences*, and the path to a state is part of the test
Missing transitions in the spec	Techniques that derive tests from stated rules cannot test a rule that was never stated	Empty cells in the table are visible, and each must be resolved as either a valid transition or an intentionally ignored event
Dead-end and trap states	No numeric boundary exists, so BVA has nothing to apply	A reachability and exit check on the graph exposes the trap
Invalid event handling	ECP invalid classes cover bad *values*, not legal values at an illegal *time*	The 45 invalid cells specify the "event in the wrong state" tests
Repetition and loops	Neither technique models cycles	1-switch sequences and loop counters test repeated reassignment
Time-driven transitions	Time is not an input field	Timeout is an explicit event (E7), and BVA is applied to its threshold
Environmental events	Connection loss and relaunch are not field values	They are modeled as events that must leave the state consistent

The relationship is complementary. ECP and BVA remain right for the *data inside* each state: address fields, tip amount, order total, promo limits. State transition testing governs the *workflow*. For a dynamic tracker, the more serious defects live in the workflow.

8. Limits and Recommendations

- The model is only as good as the states identified. Hidden states (for example, "payment captured but order not confirmed") need to be elicited from the developers and the backend, not from the UI.
- The 0-switch and 1-switch criteria do not guarantee every long sequence. The riskiest longer sequences (repeated cancellations) are added deliberately.
- The model should be stored with the requirements. A new feature (for example, "driver unreachable") means a new state or event and a table review before any code is written.
- The fix this suite demands: add the Reassigning and Failed states, a search timeout, and a rule that every waiting state has a timeout exit and a user-visible escape.

9. Expected Outcome

Measure	Before	After
States modeled	4	7
Transitions tested	About 3 (happy path)	11 of 11 valid, 45 of 45 invalid cells addressed
Mid-delivery driver cancellation	Untested	A6, A7, A8, B1, B2
Trap states	Present, unknown	Eliminated by design and verified
Dynamic scenarios	None	6
Traceability	Ad hoc	Every test maps to a state-event cell

10. Conclusion

The tracker froze because the workflow had no transition for a driver cancelling mid-delivery, and the existing tests never sent that event in that state. ECP and BVA could not reveal this, since they evaluate inputs in isolation and cannot represent history or order. Modeling the tracker as seven states and eight events exposed the missing path, and the transition table turned it into 11 valid transitions and 45 invalid cells to verify. The resulting 50 tests cover every transition, the reassignment loop and its limit, the timeout boundaries, and the dynamic cases of lost connection and relaunch. They are traceable to the model and can be automated. Used alongside ECP and BVA for field-level data, this gives complete coverage of both what users enter and what the app does next.

11.A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.

Brute Force Vulnerabilities in a Multi-Factor Authentication Module: Hybrid Error Guessing and Path Analysis

1. Failure Analysis

A brute force weakness means an attacker can make unlimited or cheap guesses against a secret. Functional testing asks “does the correct user get in and the wrong user get refused?” Brute force needs a different question: “what happens on the 5th, 50th, or 5,000th wrong attempt, and can the attacker route around the counter?” Tests that check one login at a time can never answer that.

Conceptual module (assumed policy: lock after 5 failures, OTP valid for 30 seconds, 3 OTP tries, trusted-IP list):

```
1   read user, pwd, otp, ip
2   if (!userExists(user))            return FAIL            // D1
3   if (locked(user))                 return LOCKED          // D2
4   if (!checkPassword(user, pwd))                           // D3
5       failCount[user]++
6       if (failCount[user] >= 5) lock(user)                 // D4
7       return FAIL
8   if (ipRisky(ip) || !ipKnown(ip))                         // D5 (A || B)
9       requireOtp = true
10  if (requireOtp)
11      if (otpExpired(otp))          return FAIL            // D6
12      if (!otpMatches(user, otp))   return FAIL            // D7  (no counter)
13  failCount[user] = 0
14  return SUCCESS
```

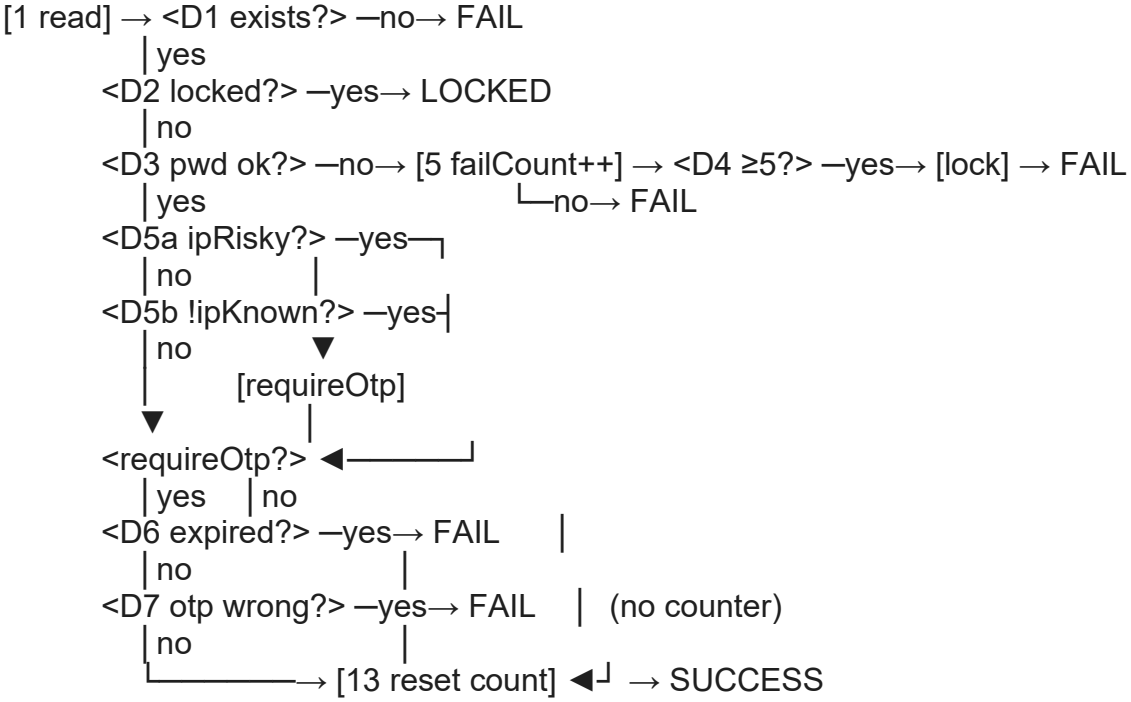
2. Critical Assessment of the Existing Integration

Weakness	How it let brute force through
Black-box tests exercised each factor once (strong password, valid OTP, trusted and untrusted IP)	Repetition was never tested, so the missing limit stayed invisible
White-box coverage reached about 100% statement and branch	Every line ran, but one execution cannot show that a line is missing a counter (line 12 has none)
Counter logic checked only for the password branch	The OTP branch, with 10^6 possible values, can be guessed without limit

Lockout keyed to the username only	Attackers rotate usernames (password spraying) or IP addresses, and the per-user counter never fires
Lockout tested for the 5th failure but not for state around it	Reset, race, and bypass behavior stayed untested
Different responses for “no such user” versus “wrong password”	User enumeration, which makes guessing cheaper
Timing differences between D1 and D3 paths	Side channel leaking whether the user exists
Tests assumed a cooperative client	No adversary behavior: parallel requests, replay, header spoofing
Security requirements weak or absent	“Reasonable protection against guessing” is not testable

The two communities were also disconnected. Black-box testers had no knowledge of line 12’s missing counter. White-box reviewers saw 100% coverage and did not look at what an attacker repeats.

3. Control Flow Graph and Path Analysis



Simple predicates: D1, D2, D3, D4, D5a, D5b, requireOtp, D6, D7 = 9, so **V(G) = 10** and at least 10 independent paths must be tested.

Path analysis for security: each FAIL exit is classified by whether it changes security state.

Path	**Exit**	**Updates attacker-visible counter?**	**Finding**
P1	Unknown user	No	Unlimited enumeration
P2	Locked	n/a	Acceptable
P3	Wrong password, count below 5	Yes	Acceptable
P4	Wrong password, count reaches 5	Yes, then locks	Acceptable
P5	Right password, expired OTP	**No**	Unlimited retries
P6	Right password, wrong OTP	No	Brute-force the OTP, the main vulnerability

P7	Success after OTP	Resets count	Lets an attacker with a valid password clear the counter
P8	Success, trusted IP	Resets count	Same concern

Paths P1, P5, and P6 are the ones standard coverage treated as "covered" because they were executed once.

4. Hybrid Strategy: Error Guessing Driven by Path Analysis

Principle: path analysis supplies a complete, finite map of where the code can go. Error guessing supplies an attacker's imagination about what to do at each point. Neither works alone: paths without adversarial intent give routine tests, and guessing without a map leaves blind spots and no measure of completeness.

Step 1: Basis path set (10 tests, mandatory)

BP	Scenario	Paths
BP1	Unknown user	D1 = F
BP2	Locked account	D2 = T
BP3	Wrong password, first failure	D3 = F, D4 = F
BP4	Wrong password, 5th failure	D4 = T
BP5	Correct password, trusted IP, no OTP	D5 both F
BP6	Correct password, risky IP, valid OTP	D5a = T
BP7	Correct password, unknown IP, valid OTP	D5b = T
BP8	Expired OTP	D6 = T
BP9	Wrong OTP	D7 = T
BP10	Full success with OTP	all pass

Step 2: Attack-oriented error-guessing catalogue attached to each path

Path	Error-guessing attack	Pass criterion
P3/P4	4, 5, 6 wrong passwords; then correct password while locked	Lock on the 5th, correct password still refused until unlock
P3/P4	4 failures, one success, 4 failures (reset abuse)	Policy defined and enforced, with no unlimited sequence
P3/P4	100 parallel wrong passwords in the same instant	Counter is atomic, not 100 attempts squeezed in before the first update
P3/P4	Case, whitespace, or Unicode variants of the username	All variants map to one counter
P1	Spray one password across 10,000 usernames	Throttling by source and by global rate, not only per user
P1 vs P3	Compare response text, status code, response size, and time	Identical for "no user" and "wrong password"
P6	1,000,000 OTP guesses in sequence	OTP attempts counted (max 3), then session and OTP invalidated
P6	Guesses against the same OTP from many connections	One shared counter per OTP
P5/P6	Replay of an accepted OTP, or an OTP from an earlier session	Single use, bound to user and session
P5	OTP at 29.9, 30.0, 30.1 seconds; client and server clock skew	Window enforced server-side with a defined tolerance
D5	X-Forwarded-For spoofing to appear as a trusted IP	Header ignored unless it comes from a trusted proxy
D5	IPv6 versus IPv4 forms of one address; rotating addresses	Address normalized, risk scored on more than IP

D5	VPN, Tor, and unreachable geolocation service	Failure of the lookup defaults to requiring OTP, never to skipping it
P7/P8	Valid password, OTP never supplied (omit the field, send null or empty)	Rejected; a missing field is not “skip”
Flow	Call the post-OTP endpoint directly without the first step	Server verifies step completion
Flow	Reuse a session token after logout or after a failed OTP	Token invalid
Lock	Attacker locks a victim out repeatedly (denial of service)	Progressive delay or CAPTCHA, with a recovery path
Lock	Unlock at the exact moment of expiry; clock change	Behaves correctly at the boundary

Step 3: Boundary values on security thresholds

Parameter	Values
Failed attempts	4, 5, 6
OTP attempts	2, 3, 4
OTP age	29s, 30s, 31s
Lock duration	just before, at, and just after expiry
Password length and strength	one below, at, and above the minimum; maximum length (to guard against long inputs)
Rate limit	N − 1, N, N + 1 requests per window

Step 4: MC/DC on the compound risk decision

For D5 (ipRisky || !ipKnown), three tests show each condition independently changes the outcome:

Test	ipRisky	!ipKnown	requireOtp
M1	F	F	No
M2	T	F	Yes
M3	F	T	Yes

A wrong operator here (|| written as &&) would skip OTP for unknown IPs, which is a bypass, so this condition is safety-critical.

Step 5: Statefulness and data flow

- Track definitions and uses of failCount, the OTP attempt counter, and session state. Every failure path must have a definition of a counter reaching a check. P6 fails this test immediately: it has no definition at all.
- Run sequences, not single calls (fail, fail, success, fail), because brute force is a property of a sequence.

Step 6: Feedback loop

Every vulnerability found by guessing is converted into a permanent path-linked regression test, and mutation testing (delete failCount++, change >= 5 to > 5, remove the OTP counter check) confirms the suite would detect those regressions. Reported vulnerabilities and any new attack idea update the catalogue.

5. Coverage Summary

Criterion	Target
Statement and branch	100%
Basis paths	10 of 10
Compound condition (D5)	MC/DC complete

Security-state check per FAIL exit	100% of exits
Threshold boundaries	below, at, above for each
Attack catalogue	Every row executed and linked to a path
Mutation score on auth logic	At least 85%
Total tests	About 10 basis + 3 MC/DC + 20 to 25 attack and boundary sequences

6. How the Approach Improves Security Posture

Concern	Contribution
Brute force on OTP	Path P6 is identified as lacking any counter, and the attack catalogue then proves the fix
Completeness	Path analysis gives a measurable map, so guessing is not a haphazard collection of ideas
Creativity	Error guessing brings attacker behavior (parallelism, spoofing, replay, spraying) that coverage criteria cannot suggest
Fail-safe design	Tests force the defaults on failure (lookup errors, missing fields) to be secure
Defense in depth	Each factor (password, OTP, IP) is tested both separately and as a chain, since attackers target the weakest one
Auditability	Paths, attacks, and boundaries map to requirements and evidence, supporting reviews such as OWASP ASVS or NIST 800-63B
Regression	Every found weakness is locked in as a test

Required design changes the tests would demand:

1. A counter on OTP attempts, with invalidation after the third failure.
2. Rate limiting by account, source, and globally, not by account only.
3. Identical responses and timing for unknown users and wrong passwords.
4. Atomic counter updates to survive parallel requests.
5. Progressive delays or step-up challenges in place of a permanent lock.
6. Server-side enforcement of flow order, with fail-closed behavior for the IP risk check.

7. Limits

- No test suite proves the absence of vulnerabilities. The strategy reduces risk in the logic it models.
- Timing, concurrency, and infrastructure-level attacks (distributed sources) need load and penetration tests, so this approach complements them and does not replace them.
- Error guessing quality depends on tester knowledge. Reviewing public attack lists and past incidents keeps the catalogue current.

8. Conclusion

Brute force vulnerabilities survived because the earlier tests checked whether each factor worked once and whether coverage numbers were high, but never examined repetition, state, or an attacker's behavior. Formal path analysis (V(G) = 10) showed which exits lacked any counter, especially the wrong-OTP path. Attack-driven error guessing, attached to each path and each threshold, supplied the adversarial cases that coverage criteria would never suggest. Combined with boundary tests, MC/DC on the IP-risk decision, sequence and data flow checks, and mutation verification, the hybrid approach gives both measurable completeness and attacker-minded depth, leaving the authentication module far more resistant to guessing attacks.

12.An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.

E-Learning Tier Upgrades on the Expiry Day: Adequacy Review and ECP Redesign

1. Failure Analysis

An upgrade on the exact day of expiration is a date-boundary defect combined with a state transition. Two rules meet on the same instant:

- Rule 1: access is valid *through* the expiry date (inclusive) or *until* it (exclusive), which the specification must state.
- Rule 2: an upgrade takes effect immediately and must replace or extend the current plan.

At the boundary the code can answer "expired" and "active" at once, so the user may be downgraded to Free, charged twice, or locked out of Premium content they just bought.

Likely causes:

Cause	Example
Inclusive/exclusive mismatch	Expiry check uses today < expiry while the upgrade check uses today <= expiry
Timestamp versus date	Expiry stored as 23:59:59 on one screen and as 00:00:00 on another
Time zone	Server in UTC, user in IST (UTC+5:30), so "today" differs for about 5.5 hours of each day
Job ordering	A nightly expiry job downgrades the account after the upgrade is recorded
Remaining-days handling	The unused day is lost or double-counted when computing the new end date
Course entitlement	Premium-only courses unlock on payment, but enrollment checks the old tier

2. Assumed Specification

Item	Rule
Tiers	Free (no expiry), Basic, Premium
Access	A paid plan is active **through the end of its expiry date** (inclusive), in the user's local time zone
Courses	Type F (free), B (Basic and above), P (Premium only)
Upgrade	Basic to Premium takes effect immediately; the new plan runs from the upgrade date for the purchased term, and unused Basic days are credited
Downgrade or lapse	After expiry, the user reverts to Free; progress is retained but B and P courses are locked
Renewal	Same-tier purchase extends from the current expiry date

3. Evaluation of Current Testing Criteria

Weakness	Effect
Dates chosen as "typical" (mid-term, long expired)	The expiry day is never used

Tiers and dates tested separately	The interaction (upgrade at the boundary) is missed
Only "active" and "expired" were treated as date states	The day-of-expiry state was never recognized as its own class
No time-of-day or time-zone variation	The intra-day edge is untested
Entitlement checked at one point only	Payment, tier record, and course access can disagree
Coverage counted as "number of scenarios"	No criterion such as all classes or all transitions

Adequacy verdict: the criteria were inadequate. Without a defined adequacy criterion (every class, every boundary, every tier-and-date combination), "enough testing" was a guess.

4. Equivalence Class Partitioning

Date classes (relative to expiry date E and today T)

ID	Class	Condition	Validity
D1	Far before expiry	$T < E - 7$	Valid
D2	Within the renewal-warning window	$E - 7 \le T < E$	Valid
D3	**Expiry day**	$T = E$	Valid, special
D4	Within the grace period (if any)	$E < T \le E + g$	Valid
D5	Long after expiry	$T > E + g$	Valid
D6	No expiry date (Free tier)	E is null	Valid
D7	Invalid date	E before the start date, a malformed value, or a future start	Invalid

D3 is its own class because behavior changes there. Treating "today = expiry" as part of "active" is exactly the thinking that produced the defect.

Subscription classes

ID	Current tier	Purchase action
S1	Free	Upgrade to Basic
S2	Free	Upgrade to Premium
S3	Basic	Upgrade to Premium
S4	Basic	Renew Basic
S5	Premium	Renew Premium
S6	Premium	Downgrade to Basic
S7	Premium or Basic	No action (lapse)
S8	Any	Invalid (unknown tier, duplicate purchase, payment failure)

Course classes

ID	Course type	Required tier
C1	Free course	Any
C2	Basic course	Basic or above
C3	Premium course	Premium

Time-of-day classes (for the expiry day only)

ID	Class
T1	Start of the day (00:00:00 to 00:00:01)
T2	Mid-day

T3	Last second (23:59:59)
T4	Just after midnight (00:00:00 the next day)
T5	Time-zone offset case: user's date differs from the server's date

5. Boundary Values on the Dates

Boundary	Test dates
Start of the warning window	E − 8, E − 7, E − 6
Expiry	E − 1, **E**, E + 1
End of the grace period	E + g − 1, E + g, E + g + 1
Term length	purchase on day 1, last day, one day past
Leap-year and month-end	E = 28 Feb, 29 Feb, 31 Jan, 31 Dec to 1 Jan

6. Structured Test Design

Valid combinations are built from classes by crossing the date class with the purchase action, then removing infeasible pairs (for example, Free has no expiry, so D1 to D5 do not apply to it).

Layer A: Expiry-day upgrade (the reported defect), 12 tests

Today = E in every row.

TC	Current plan	Time	Action	Expected
A1	Basic	00:00:01	Upgrade to Premium	Premium active immediately, expiry recalculated, P-courses open
A2	Basic	12:00	Upgrade to Premium	Same as A1, no downgrade by the nightly job
A3	Basic	23:59:59	Upgrade to Premium	Same, and the new term is not shortened by a day
A4	Basic	00:00:00 (E + 1)	Upgrade to Premium	Treated as an expired user: Free, then new Premium, with no credit for unused days
A5	Basic	12:00	Renew Basic	New expiry = E + term (extended from E, no gap, no overlap)
A6	Premium	12:00	Renew Premium	Extension from E
A7	Premium	12:00	Downgrade to Basic	Per policy: effective at expiry or immediately, but access is never lost
A8	Basic	12:00	No action	Access through 23:59:59, then Free from E + 1
A9	Basic	12:00	Upgrade, payment fails	Remains Basic until the end of the day, a retry is possible
A10	Basic	12:00	Upgrade twice (double click)	One charge, one upgrade
A11	Basic	User's time zone is E, server's is E + 1	Upgrade	Decision uses one defined time zone, with a consistent result
A12	Basic	23:59:59 server, payment confirmed 00:00:02	Upgrade	No lost days; treated by the time of order, not the time of confirmation (per policy)

Layer B: Date-class × tier access matrix (what a user can open)

TC	Tier	Date class	Course type	Expected
B1	Basic	D1	C2	Open
B2	Basic	D2	C2	Open, renewal banner

B3	Basic	D3	C2	Open (inclusive)
B4	Basic	D4	C2	Per grace policy
B5	Basic	D5	C2	Locked, upgrade prompt
B6	Basic	D3	C3	Locked, upgrade prompt
B7	Premium	D3	C3	Open
B8	Premium	D5	C3	Locked
B9	Premium	D5	C1	Open (free course, any tier)
B10	Free	D6	C1	Open
B11	Free	D6	C2	Locked
B12	Free	D6	C3	Locked

Layer C: Upgrade transitions away from the boundary (to separate the edge from general upgrade logic)

TC	Current	Date class	Action	Expected
C1	Free	D6	To Basic	Basic active, expiry = today + term
C2	Free	D6	To Premium	Premium active
C3	Basic	D1	To Premium	Credit for unused days, new expiry computed
C4	Basic	D2	To Premium	Same, with a warning-window banner cleared
C5	Basic	D4	To Premium	Reinstated plus upgrade, no double credit
C6	Basic	D5	To Premium	Treated as a new purchase, no credit
C7	Premium	D1	To Basic	Policy-defined, no access loss before the end of term

Layer D: Invalid classes

TC	Input	Expected
D01	Expiry date earlier than the start date	Rejected or flagged
D02	Unknown tier code	Rejected
D03	Malformed date	Rejected
D04	Purchase for the tier the user already holds, on the same day, twice	Idempotent, not stacked
D05	Free user “renews”	Rejected or treated as a new purchase

Layer E: Calendar edges

TC	Scenario	Expected
E1	E = 28 Feb in a non-leap year, upgrade on E	New expiry computed correctly across 1 Mar
E2	E = 29 Feb in a leap year	Valid, no crash
E3	E = 31 Dec, upgrade on E	Year rollover is correct
E4	One-month term bought on 31 Jan	End date follows the stated month-end rule
E5	Daylight-saving or time-zone shift on E	Day length does not alter validity

Layer F: Consistency checks after every upgrade (asserted in all tests above)

Check	Systems that must agree
Subscription record	Tier and expiry match
Payment ledger	One charge, correct amount, with credit applied
Entitlement service	P/B/F course access matches the tier
Session	Existing logged-in sessions refresh, with no stale token
Notifications	No “your plan has expired” email after a same-day upgrade
Scheduled job	The expiry job skips users who upgraded that day

Total: 12 + 12 + 7 + 5 + 5 = 41 tests, with Layer F assertions applied across all of them.

7. Justification of Completeness

Completeness here is claimed against stated criteria, not as a general proof.

Criterion	Result
Every date class D1 to D7	Covered (Layers A to D)
Every subscription class S1 to S8	Covered (A, C, D)
Every course type C1 to C3 against every relevant tier	Covered (B)
Expiry day across every time-of-day class T1 to T5	Covered (A1 to A4, A11, A12)
Boundaries: E − 1, E, E + 1, and the grace-period edges	Covered
Calendar edges (leap, month-end, year-end)	Covered (E)
Invalid inputs	Covered (D)
Cross-system consistency	Asserted in every test (F)

Reduction: with 7 date classes, 8 actions, 3 course types and 5 time-of-day classes, there are 7 × 8 × 3 × 5 = 840 combinations. Removing infeasible pairs and using one representative per class leaves 41 tests, a reduction of about 95%.

Why this ensures seamless transitions:

- The expiry day, the source of the defect, is given its own class and tested at several times of day instead of being absorbed into “active”.
- Upgrade tests are run on both sides of the boundary (Layer C against Layer A), so a fix that merely moves the error to the day before or after is detected.
- Consistency checks catch cases where the UI shows Premium but the entitlement service still returns Basic.
- Idempotency and failure cases cover real-world retries and payment errors that occur during upgrades.

8. Limits and Required Clarifications

- The specification must state whether the expiry is inclusive, which time zone governs “today”, and how unused days are credited. If those answers are missing, the test oracle itself is ambiguous and the requirement should be fixed first.
- ECP assumes all values in a class behave alike. The time-zone and clock-based behavior may break this assumption, which is why the time-of-day classes were added.
- Concurrency (an upgrade racing with the nightly job) and load behavior need separate tests, since ECP does not model timing.

9. Conclusion

The expiry-day failures occurred because the test design treated dates as “active” or “expired” and never recognized the expiry day as a separate equivalence class where inclusive limits, time zones and background jobs collide. Partitioning dates into seven classes, subscriptions into eight, and courses into three, with time-of-day sub-classes for the critical day, exposes every combination that matters. The 41-test suite checks upgrade behavior at, before and after the boundary, validates consistency between billing, entitlement and the user interface, and handles calendar edge cases. Once the specification fixes the inclusive rule and the time zone, this suite gives traceable and complete coverage of the transitions users actually experience.

13.A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.

Smart-Home Lighting Controller: Beyond Statement Coverage

1. Risk Evaluation: Why Statement Coverage Alone Is Dangerous

Statement coverage confirms that each line ran once. In a controller built from switch statements and compound Boolean expressions, most of the logic lives in places where a line can run without the logic being tested.

Conceptual module (assumed: lux sensor 0 to 1000, modes AUTO, MANUAL, NIGHT, VACATION, hysteresis to avoid flicker):

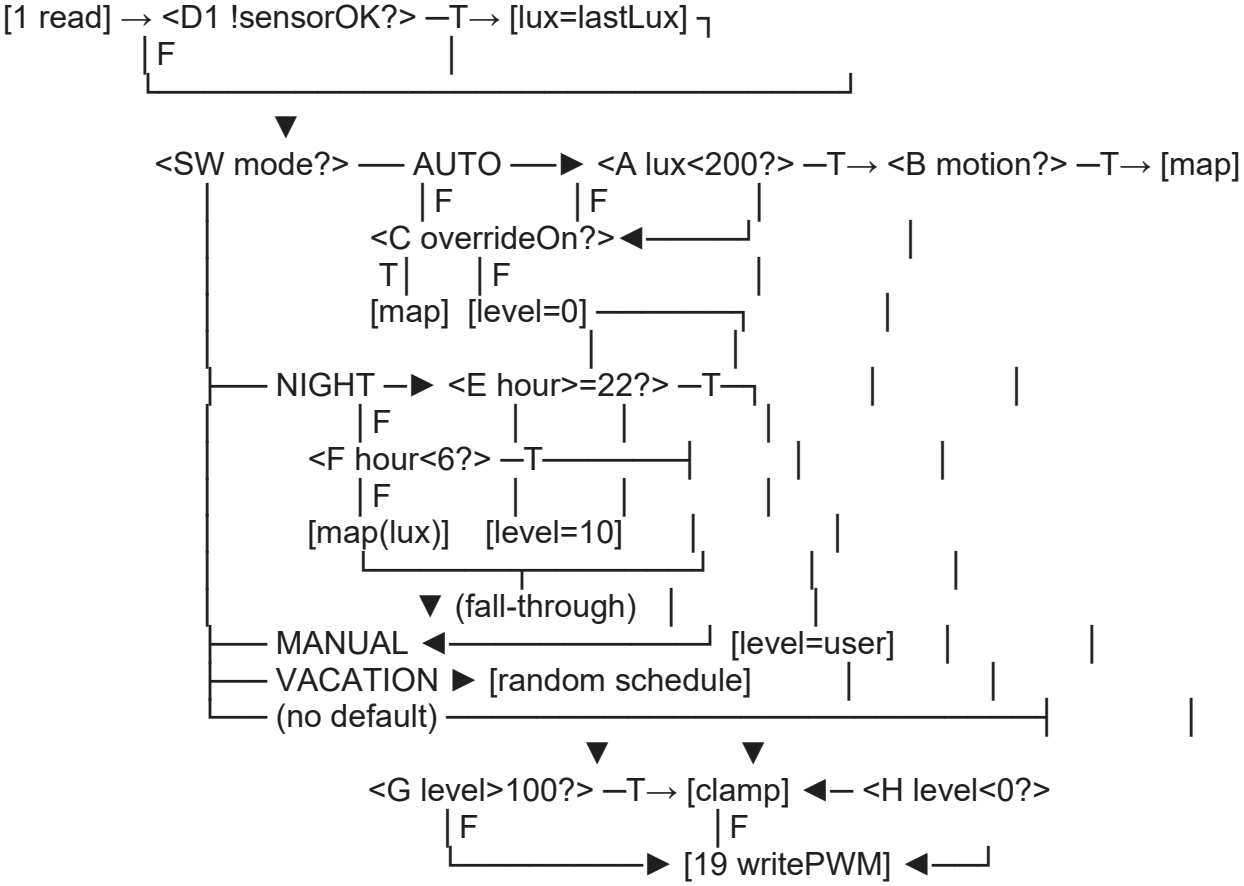
```
1   read lux, mode, motion, hour, sensorOK, overrideOn
2   if (!sensorOK) lux = lastLux                          // D1
3   switch (mode)                                         // SW
4     case AUTO:
5       if ((lux < 200 && motion) || overrideOn)          // D2: (A && B) || C
6           level = map(lux)
7       else level = 0
8       break
9     case NIGHT:
10      if (hour >= 22 || hour < 6) level = 10            // D3: E || F
11      else level = map(lux)
12      // missing break: falls through
13    case MANUAL:
14      level = userLevel; break
15    case VACATION:
16      level = randomSchedule(hour); break
17    // no default
18  if (level > 100 || level < 0) level = clamp(level)    // D4: G || H
19  writePWM(level)
```

Risk of relying on statement coverage	**What it hides here**
Every line can run with one test per case	Fall-through from NIGHT into MANUAL (line 12) is not a statement, so it is invisible. NIGHT's level is silently overwritten by userLevel.
No default branch exists	An unexpected mode value (memory corruption, firmware mismatch, a new app version sending mode 5) executes nothing and leaves level uninitialized or stale. No statement is "missed", so coverage stays 100%.
if without else (line 2, line 18)	The false outcome has no statement of its own, so it can go untested.
Compound conditions	One test makes `(A && B)
Clamp reached by one input	The level > 100 side runs, level < 0 never does, and the statement still counts as covered.

Hardware-dependent values	Sensor saturation, zero, or noise never appear in nominal tests.

For hardware, the cost of a gap is physical: flickering, full-brightness lights at night, a PWM write of an out-of-range value that can damage an LED driver, or lights that ignore the user. These are failures at the software-hardware seam, which a unit test with mocked ideal inputs does not reach.

2. Control Flow Graph and Complexity



Simple predicates: D1, A, B, C, E, F, G, H = 8. Switch outcomes: 4 cases + implicit default = 5 (contributing 4 extra decision edges). Counting each case as a predicate, **V(G) = 8 + 4 + 1 = 13**. At least 13 independent paths exist.

3. Branch Coverage: Gaps Revealed

Assume the existing suite had five tests: one per mode with a normal sensor and sensible inputs.

Decision	Outcomes	Covered by existing suite	Missing
D1	T, F	F only	Sensor failure
SW	AUTO, NIGHT, MANUAL, VACATION, default	4 of 5	**default**, and NIGHT-without-fall-through (infeasible as written)
D2	T, F	T, F	none
D3	T, F	T only (tested at night)	Daytime NIGHT-mode
D4	T, F	F only	Clamp never taken

Branch coverage: 7 of 13 outcomes, about 54%, against statement coverage near 90%. The gap between the two numbers is the measure of what statement coverage hid.

Logical paths identified as missing:

1. Unknown mode value (no default)

2. Sensor failure with a stale lastLux
3. NIGHT mode in daylight hours
4. NIGHT mode that falls through into MANUAL, which exposes a probable defect (a missing break) that only a deliberate test with an assertion on the NIGHT output notices
5. Clamp on both sides (>100 and <0)

4. Condition Coverage: Gaps in Compound Expressions

Branch coverage needs each decision to be true and false. Condition coverage requires each *atomic condition* to be true and false, and MC/DC adds that each one must independently affect the result.

D2: (A && B) || C, where A = lux < 200, B = motion, C = overrideOn.

Existing tests: (A=T, B=T, C=F) giving true and (A=F, B=F, C=F) giving false. Branch coverage is satisfied. Condition coverage is nearly satisfied, but C is never true, so the override path was never exercised at all, and B's independent effect was never shown.

Test	A	B	C	Outcome	Independence shown
M1	T	T	F	T	base
M2	F	T	F	F	A (vs M1)
M3	T	F	F	F	B (vs M1)
M4	T	F	T	T	C (vs M3)

Four tests give MC/DC for D2.

D3: E || F (hour ≥ 22 or hour < 6)

Test	E (hour ≥ 22)	F (hour < 6)	Outcome
N1	F	F	F (hour 12)
N2	T	F	T (hour 22 or 23)
N3	F	T	T (hour 0 to 5)

A typical test at hour 23 sets only E. F is never exercised, so a defect such as hour < 6 written hour < 0 survives.

D4: G || H needs (F,F), (T,F), (F,T): level 50, 150 and −5.

Compiler caveat: with short-circuit evaluation, B is not evaluated when A is false. Condition coverage must be measured on the instrumented build, because the number of reachable combinations differs from the truth table.

5. Improved White-Box Strategy

Layer 1: Coverage gates

Criterion	Target
Statement	100% (baseline)
Branch, including every case and the implicit default	100%
Condition	100%
MC/DC on D2, D3, D4	100%
Basis paths	13 of 13

Layer 2: Concrete test set

TC	Focus	Inputs	Expected

T1	Sensor failure	sensorOK = F, lastLux = 150	Uses stale value, flags a sensor warning
T2	Sensor failure with no history	sensorOK = F, lastLux unset	Safe default, no uninitialized read
T3 to T6	MC/DC for D2	M1 to M4	As tabulated
T7	NIGHT at hour 12	mode = NIGHT, hour = 12	map(lux), no override by userLevel
T8	NIGHT at hour 23	N2	level = 10
T9	NIGHT at hour 3	N3	level = 10
T10	NIGHT output assertion	as T8	Level stays 10, **fails today** because of the fall-through into MANUAL
T11	MANUAL	userLevel = 60	60
T12	VACATION	hour varies	Schedule output within 0 to 100
T13	Unknown mode	mode = 7	Safe state defined by the requirements (for example, hold the last level), no uninitialized level
T14	Clamp high	level = 150	100
T15	Clamp low	level = −5	0
T16	Clamp none	level = 50	50

Layer 3: Boundary values on thresholds

Parameter	Values
lux threshold	199, 200, 201
Hour boundaries	5, 6 and 21, 22
Level limits	−1, 0, 100, 101
Sensor range	0, 1, 999, 1000, 1001 (saturation and overrange)

Layer 4: Stateful and time-based tests

Dimming controllers have memory (lastLux, hysteresis, ramp).

Test	Sequence	Expected
S1	lux oscillating around 200, ±2	No flicker (hysteresis holds)
S2	mode change AUTO to NIGHT to AUTO	No stale level carried across
S3	motion on, off, on within the timeout	Timer resets correctly
S4	sensor fails for N cycles, recovers	Smooth return, no jump
S5	override on, then off	Falls back to AUTO logic

Layer 5: Data flow

Check that level is defined on **every** path before writePWM(level). The unknown-mode path (no default) shows a use with no definition, which this check flags directly.

Layer 6: Mutation testing

Mutants: delete a break, change && to ||, < to <=, 200 to 199, remove the clamp. Require a kill rate of at least 85%. Surviving mutants mark assertions too weak (for example, tests that run the NIGHT case without asserting its output).

Layer 7: Hardware-in-the-loop (HIL) and fault injection

Replace software mocks with a sensor simulator or real hardware on the bench:

- ADC noise, saturation, stuck-at values, and dropouts
- PWM output read back to confirm the duty cycle matches level
- I²C or ADC read timeouts and bus errors

- Power brown-out and reboot mid-ramp, to verify state recovery
- Timing: confirm the control loop meets its period under maximum load

6. How This Reduces Hardware-Software Integration Risk

Integration risk	How the strategy addresses it
Out-of-range value reaches PWM and stresses the driver	Clamp tests on both sides (T14, T15), data flow check that level is always assigned, HIL read-back
Unknown mode from the app or a firmware mismatch	Default-branch test (T13) forces a defined safe behavior
Sensor failure, noise, or saturation	T1, T2 and boundary tests at 0, 1000 and 1001, plus fault injection
Silent logic overrides (fall-through)	Branch coverage plus T10 asserts the output, exposing the missing break
Override and motion conditions that never mattered in tests	MC/DC shows each condition independently changes the output
Flicker and instability from threshold edges	199/200/201 tests and hysteresis sequences
Behavior that depends on history	Stateful sequences S1 to S5
Mocks that hide hardware timing	HIL tests the real interface, which unit-level coverage cannot do
Weak assertions	Mutation scoring

Measure	Before	After
Statement	about 90%	100%
Branch	about 54%	100%
Condition and MC/DC	partial	100% on D2, D3, D4
Basis paths	few of 13	13 of 13
Default and failure paths	untested	T1, T2, T13
Hardware seam	mocks only	HIL and fault injection
Suite size	5	about 16 logic tests plus about 12 boundary, 5 sequence, HIL scenarios

7. Justification and Limits

- **Why branch coverage first:** it forces every case, the default, and both sides of every if to execute, which removes the biggest hole in statement coverage.
- **Why MC/DC on the Boolean expressions:** it grows linearly (n + 1 tests per decision) and detects wrong operators and masked conditions, the typical faults in sensor logic.
- **Why not path coverage:** the stateful loop makes full path testing unbounded, so V(G) basis paths plus targeted sequences give finite, defensible coverage.
- **Limits:** 100% MC/DC does not prove correct physical behavior. Hardware timing, electrical faults and sensor drift need HIL and field testing, and the missing default behavior is a requirements gap that should return to the design owner, not only to the tests.

8. Conclusion

Statement coverage would have approved this module while hiding the most dangerous behavior: a switch with a fall-through and no default, a clamp that never runs, and Boolean expressions where override and motion never mattered. Branch coverage exposes the missing outcomes (about 54% against a reported 90%), and condition coverage with MC/DC shows which atomic conditions were never tested independently. Adding boundary, stateful, data flow and mutation checks, and then validating on hardware with fault injection, makes the logic provably exercised and the software-

hardware interface realistically stressed. The result is a controller that fails safe, instead of one whose gaps appear only when a real sensor misbehaves.

14.A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.

Zero and Negative Weights in a Shipping Cost Calculator: Robustness BVA

1. Failure Analysis

Shipping cost is a function of three inputs. When weight is 0 or negative, the system either produces a nonsense price (a negative or zero charge) or crashes. Both outcomes indicate that input validation was never specified as a testable behavior, so testers checked only that valid shipments price correctly.

Assumed specification (none given, so I fix one):

Item	Rule
Weight	0.1 kg to 500 kg, up to 1 decimal place
Zone	Z1 local, Z2 regional, Z3 national, Z4 international
Speed	Standard, Express, Same-day (Same-day only for Z1 and weight ≤ 20 kg)
Cost	Base rate per zone + per-kg rate × weight, times a speed multiplier (1.0, 1.5, 2.5)
Invalid weight	Must be rejected with a clear message, and no price or order is created

Why zero and negative weights slip through:

Cause	Consequence
Validation checks only the upper limit (weight > 500)	0 and −5 pass the check
Pricing formula accepts any number	Negative weight gives a negative per-kg charge that cancels the base rate, so the customer may be paid to ship
Weight 0 treated as “not entered” or as a valid light parcel	Zero-cost or base-only shipment is created
Validation only in the UI	The API or a bulk import accepts the value directly
Rounding before validation	−0.04 rounds to 0.0 and slips past a weight < 0 check
Type handling	“-0”, “−5” (Unicode minus), “1e-3” parsed differently

2. Why Current Input Validation Testing Failed

Weakness	Effect
Only valid classes were designed (“typical” weights such as 2, 10, 50 kg)	Invalid space never explored
Standard BVA applied to valid edges only (0.1 and 500)	The first invalid value below the range was assumed to behave like any invalid value, and was never run
Negative scenarios treated as optional	Time was spent on happy paths
Expected behavior for invalid input not defined	Testers had no oracle for “what should happen at −1”
Weight tested alone	The interaction with zone and speed (for example, negative weight with Express) was missed
Only the UI was tested	The API and import paths remained open
No check of side effects	A rejected input might still leave a draft order, a log error or a charge

3. Robustness Boundary Value Analysis

Standard BVA tests values at the boundary of the valid range. **Robustness testing** extends it to the first value *outside* the range on each side, and then to the extreme values beyond.

3.1 Weight boundary values

Region	Value	Expected
Far below	−1,000,000	Reject
Far below	−0.1	Reject
Just below zero	−0.1 (smallest negative step)	Reject
Zero	0	Reject (below the 0.1 minimum)
Just below minimum	0.0	Reject
Sub-resolution	0.04, 0.05	Reject or defined rounding, since 0.04 is below the minimum
Minimum (valid)	0.1	Accept
Just above minimum	0.2	Accept
Nominal	25.0	Accept
Just below maximum	499.9	Accept
Maximum (valid)	500.0	Accept
Just above maximum	500.1	Reject
Far above	99,999,999	Reject, no overflow

The robustness points are the first, third and fourth rows of invalids and the last row. Standard BVA would never run them.

3.2 Non-numeric and special values (robustness beyond numbers)

Input	Expected
Blank / null / missing field in API	Reject, “weight required”
“abc”, “5kg”, “5,0”	Reject, format error
“−5” with a Unicode minus	Reject, not silently parsed as 5
“-0”, “0.0”, “00.0”	Reject (equal to zero)
“+5”	Defined behavior, consistent across UI and API
“1e2”, “1e-3”	Reject or parse per spec, never wrong silently
NaN, Infinity, −Infinity	Reject
0.1 + 0.2 style floating results (0.30000000000000004)	Handled by decimal arithmetic, not by raw float comparison
Whitespace “ 5 “	Trim or reject, consistently
Very long string (10,000 digits)	Reject, no crash
Integer overflow (2,147,483,648)	Reject, no wrap-around to negative

The overflow row shows why negative scenarios matter even for “positive” input: a large positive value can wrap into a negative one.

4. Combining Invalid Weight with Other Fields

Invalid weight must be tested **with each zone and speed**, because validation order can mask or expose errors.

TC	Weight	Zone	Speed	Expected
N01	0	Z1	Standard	Reject, weight error only
N02	−5	Z3	Express	Reject, no negative price shown

N03	−0.1	Z4	Same-day	Reject; the weight error is shown first, or both errors are reported consistently
N04	0	Z2	Express	Reject, no base-only price
N05	−5	none selected	none selected	Reject, all missing and invalid fields reported
N06	600	Z1	Same-day	Reject, over the maximum and over the 20 kg same-day limit
N07	0.1	Z4	Same-day	Reject for the combination rule, not for the weight
N08	20.1	Z1	Same-day	Reject (same-day limit, a valid weight)

N06 and N07 show why single-field tests are insufficient: the right error message depends on the combination.

Valid control cases, so a fix that rejects too much is also caught:

TC	Weight	Zone	Speed	Expected cost
V01	0.1	Z1	Standard	Base Z1 + 0.1 × rate
V02	500.0	Z4	Standard	Maximum price, no overflow
V03	20.0	Z1	Same-day	Accepted
V04	25.0	Z3	Express	Correct total

5. Verifying Rejection Properly

A rejected input needs more than a visible error message. For every negative test, assert:

Check	Why
Error message text and field highlighted	The user can fix the input
HTTP status (for the API) is a client error such as 400 or 422, not 200 and not 500	500 would mean the server crashed instead of validating
No price calculated or displayed	No negative or base-only quote
No order, draft, or payment record created	No side effects
Entered data in other fields preserved	Usability
No stack trace or internal detail exposed	Security
Event logged for monitoring	Detects abuse or integration bugs
Same result via UI, API, and bulk import	No unprotected entry point

6. Test Suite Summary

Group	Count
Weight boundary values (valid and invalid)	13
Non-numeric and special inputs	11
Invalid weight × zone × speed	8
Valid control cases	4
Entry-point checks (UI, API, import)	3
Total	**39**

The full input space is 4 zones × 3 speeds × unbounded weights. The suite reduces it by treating each invalid weight class as one representative per entry point, and keeping zone and speed combinations only where a rule interacts (for example, same-day).

7. Justification: Why Negative Scenarios Are Necessary

Reason	Explanation
Defects concentrate at the edges	Programmers write the success path carefully and handle invalid input as an afterthought. Failures cluster where the code has not been thought through.

Financial integrity	A negative weight can produce a negative or zero charge. The business loses revenue directly, or a customer is credited.
Real users and systems send bad data	Typing mistakes, copy-paste errors, integrations with different units, and scripts all produce zero and negative values. They are normal operating conditions, not rare ones.
Security	Systems that accept out-of-range numbers can be manipulated (price tampering through API parameters). Negative testing is a basic check against input abuse.
Robustness	Software must degrade gracefully. A 500 error or a crash on bad input shows the system cannot be relied on under unexpected conditions.
Error propagation	An invalid weight accepted at checkout corrupts downstream systems: invoicing, carrier labels, route planning, inventory. One bad record causes many later failures.
Specification quality	Writing the negative tests forces the question “what should happen here?”, which often exposes missing requirements.
Positive tests cannot find it	A suite that passes every valid input tells you nothing about invalid input. Only a deliberate negative test can show validation is absent.

Why robustness BVA specifically: standard BVA assumes the first invalid value fails correctly. In practice the first value past the edge (500.1, 0.0, −0.1) is exactly where > versus >= and rounding errors live, so it must be run. Extreme values then test overflow, type, and parsing, which are different failure classes.

8. Limits and Recommendations

- The suite verifies the specified limits. If the true business limits are not agreed (for example, whether 0.05 kg rounds up to 0.1), the oracle itself is uncertain, and the requirement should be fixed first.
- Add validation in one shared server-side function used by the UI, API, and import, and test it as a unit as well as end to end, so a gap cannot reappear in one channel.
- Convert every production incident involving bad input into a permanent regression case.
- Complement with fuzz testing to find malformed inputs that no one thought to list.

9. Conclusion

Zero and negative weights escaped because the test design looked only at valid ranges and treated invalid input as an afterthought, with no defined expected outcome and no testing outside the UI. Robustness BVA fixes this by testing the first invalid values on both sides (0, −0.1, 500.1), the extremes beyond them, and the special and non-numeric forms where parsing and overflow fail. Combining those values with zones and speeds, and checking that rejection leaves no price, order, or stack trace behind, gives 39 traceable tests. Negative scenarios are necessary because they protect revenue, data integrity, and security, and because no number of passing valid-input tests can show that invalid input is handled safely.

15.A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.

Prime-Sized Datasets in a Parallel Imaging Module: Why Random Testing Failed and a Basis Path Redesign

1. Failure Analysis

Results differ only when the data size N is prime. That strongly suggests a work-partitioning defect. A prime N has no divisors other than 1 and itself, so it cannot be split evenly across any thread count T > 1. The remainder N mod T is never 0, and the code that handles the leftover elements is the only place where prime sizes behave differently from most composite ones.

Conceptual module (assumed: T worker threads, chunked loop, partial results merged into a final sum or filter output):

```
1   chunk = N / T                          // integer division
2   rem   = N % T
3   if (N <= 0) return ERROR                    // D1
4   if (N < T) T = N                       // D2  fewer items than threads
5   for t in 0..T-1 (parallel):             // loop L1
6       start = t * chunk
7       end   = start + chunk
8       if (t == T-1 && rem > 0) end = N        // D3 (P && Q)  remainder handling
9       for i in start..end-1:                 // loop L2
10          if (mask[i]) partial[t] += f(data[i])   // D4
11  total = 0
12  for t in 0..T-1: total += partial[t]        // loop L3  merge
13  if (N % 2 == 1) total = normalize(total)       // D5  odd-size correction
14  return total
```

Where prime sizes break it:

Fault	Why only prime (or near-prime) N exposes it
Lines 1 and 4 use chunk = N / T before T is reduced	When N < T, chunk becomes 0 and no element is processed
Line 8 gives the whole remainder to the last thread	Its range may be larger than the others, so it may overrun the buffer or take much longer, changing floating-point accumulation order
Remainder handled on a condition that is false for sizes divisible by T	Composite sizes that divide evenly never enter the branch
Line 13's odd-size normalization	Every prime except 2 is odd, so the branch runs in nearly every prime test and is rarely compared against an exact reference
partial[t] not zero-initialized or shared without a barrier	A race becomes visible only when the last thread runs longer than the others, which happens exactly when rem > 0
Floating-point summation order	Different chunk sizes change rounding, so two runs differ in low-order bits

The defect is therefore a structural combination (remainder branch, odd-size branch, uneven thread load), not a numeric range problem.

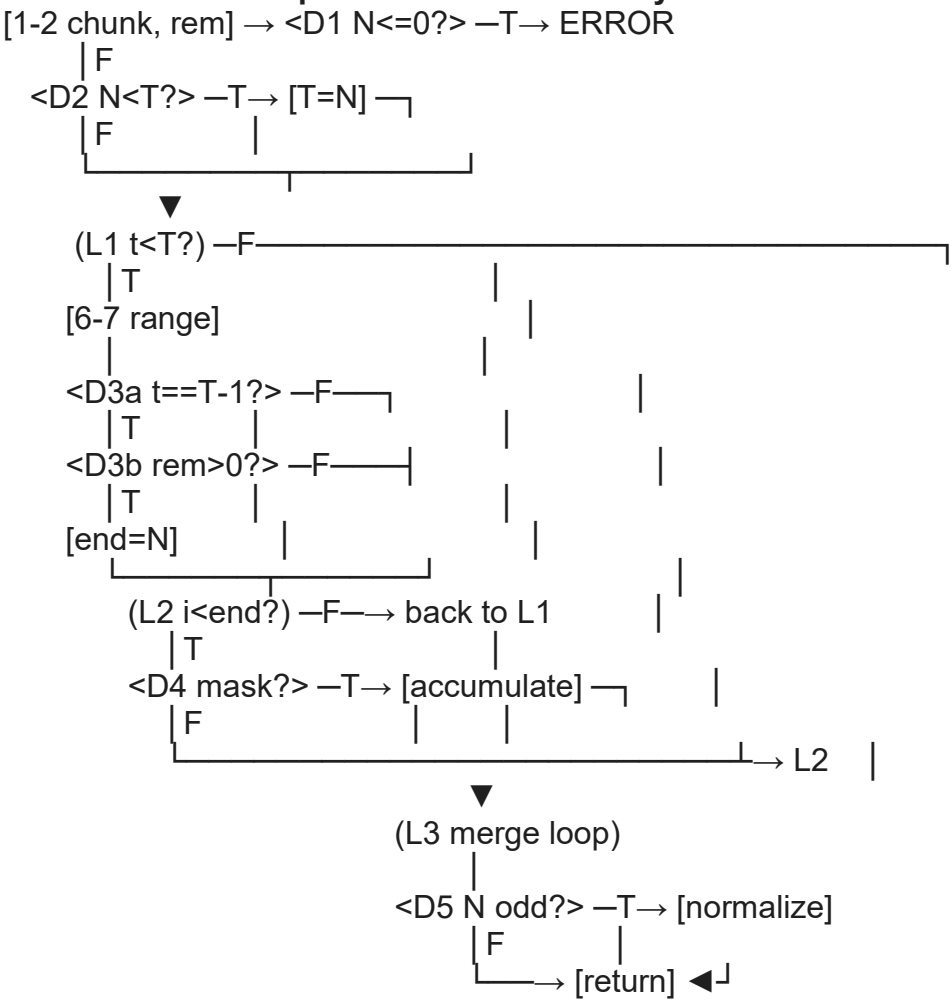
2. Evaluation of Reliance on Random Testing

Random sizes seem broad, but they are poorly matched to the defect.

Issue	Effect here
Primes are sparse. Near 10^6, about 1 in 14 integers is prime	Most random sizes exercise the even-split path, so the faulty branch is hit only occasionally
Sizes are drawn uniformly, not by structure	Special classes such as N = 1, N < T, N = T, N = T ± 1, N = 2^k ± 1 are nearly never chosen
Executions are non-repeatable	A race or ordering fault may show once in thousands of runs, and the failing seed is not logged
No oracle for numeric correctness	A random test can only say “no crash”; it cannot say the sum is right
No coverage measurement	Nobody can state which branches or paths the random sizes reached
Parallel nondeterminism	The same input can pass 999 times and fail once, so one passing run proves almost nothing
Tolerance hides drift	A loose comparison threshold passes results that differ in the 6th digit
Silent in medical context	An inconsistent pixel sum does not raise an error; it changes a measurement a clinician may rely on

Verdict: random testing is a useful supplement for input diversity and for shaking out unexpected crashes, but it cannot be the primary strategy for a module with discrete structural conditions (remainder, odd size, N < T) and a safety consequence. It samples the input space. The defect lives in the code structure.

3. Control Flow Graph and Basis Path Analysis



Simple predicates: D1, D2, L1, D3a, D3b, L2, D4, L3, D5 = 9.
V(G) = 9 + 1 = 10, so at least 10 independent paths must be exercised.

4. Structured White-Box Strategy

Layer 1: Basis path set (10 tests)

Use a fixed thread count T = 4 for determinism in this layer.

BP	N	Mask	Path exercised	Purpose
BP1	0	–	D1 = T	Error exit
BP2	−5	–	D1 = T	Negative size
BP3	3	all true	D2 = T (N < T)	Thread reduction, chunk would be 0
BP4	8	all true	D2 = F, D3b = F, D5 = F	Even split, even size (baseline)
BP5	7	all true	D3b = T, D5 = T	**Prime, remainder 3, odd: the reported failure**
BP6	9	all true	D3b = T, D5 = T	Composite odd with remainder, to separate "prime" from "has remainder"
BP7	10	all true	D3b = T, D5 = F	Even size with remainder, to isolate the odd-size branch
BP8	9	all false	D4 = F every iteration	Empty accumulation, partial stays 0
BP9	9	alternating	D4 mixed	Mask handling in the remainder range
BP10	1	true	Minimal size, T reduced to 1	Single-element path

BP5, BP6 and BP7 are the key set. If BP5 fails and BP6 passes, the fault depends on primality or a related structural feature. If BP6 and BP7 fail as well, the real cause is the remainder branch or odd-size correction, not primality. That separation is the main analytic benefit of path-based selection.

Layer 2: Loop boundary testing

Loop	Values
L2 (elements per chunk)	0, 1, 2, typical, chunk − 1, chunk, chunk + 1
L1 (threads)	T = 1, 2, typical, hardware max, max + 1 (oversubscription)
Combined	N = T − 1, T, T + 1, 2T − 1, 2T, 2T + 1 for each tested T
Remainder	rem = 0, 1, T − 1 (the last thread receives the largest extra range)

Layer 3: Prime and structure-aware size classes

Because primality itself may matter through divisibility, deliberately include:

Class	Example N
Small primes	2, 3, 5, 7, 11, 13
Large primes	1,009; 65,537; 1,000,003
Prime near a power of two	127, 257, 8,191
Primes of the form T ± 1	for T = 4: 3, 5; for T = 8: 7
Composite controls with the same remainder	9, 15, 21
Powers of two and ±1	64, 63, 65
Maximum supported size and max + 1	per specification

Each prime is paired with a composite of similar magnitude and the same N mod T, so a result difference can be attributed to the correct cause.

Layer 4: Mathematical oracle

Correctness is checked, not just stability.

Check	Method
Reference result	A single-threaded, serial implementation using high-precision or compensated (Kahan) summation

Tolerance	Relative error bound derived from the data type, for example below 1e-6 for float32 sums, not an arbitrary loose threshold
Order independence	Result with T = 1, 2, 4, 8 must agree within the bound
Invariants	Sum of a constant array equals constant × N; sum of a zero mask is exactly 0; adding an element changes the result by exactly f(x)
Metamorphic tests	Shuffling the input preserves the result; processing N elements as two halves and merging equals processing all N
Exact data	Integer-valued inputs make floating-point sums exact, so any difference signals a logic fault, not rounding

Layer 5: Concurrency and stability tests

Test	Targets
Repeat each basis test 1,000 times with the same inputs	Intermittent races (a pass is not proof, so use volume)
Thread sanitizer and race detector on all basis tests	Unsynchronized writes to partial[t] or the shared total
Barrier check: delay the last thread artificially	Merge starting before all partials are ready
Bounds checking enabled (address sanitizer)	Last thread overrunning the buffer when end = N
Uninitialized memory detector	partial[t] not zeroed
Fixed seeds and a deterministic reduction order	Bitwise reproducibility, or a documented tolerance
Memory stress at the largest dataset	Allocation failure handled, no partial results returned
Thread-count changes between runs	Result unaffected

Layer 6: Data flow and mutation testing

- **Data flow:** every definition of partial[t] must reach the merge, and end must be defined on every path before use. This flags the missing zero-initialization and the unreduced chunk after line 4.
- **Mutation testing:** change chunk = N / T to (N + T - 1) / T; change rem > 0 to rem >= 0; remove the odd-size normalization; change N < T to N <= T; delete the barrier. Require a kill rate of at least 85%. A surviving mutant means an assertion is too weak.

Layer 7: Random testing, retained in a supporting role

Run seeded property-based tests (for example, random N with the serial reference as oracle) after the structured layers. The seed is logged on every run, and any failure is converted into a permanent named test case. Random testing then discovers unanticipated structure, rather than carrying the coverage burden.

5. Justification

Concern	How the approach addresses it
Fault is conditional on structure	Basis paths include the remainder branch and odd-size branch by construction (BP5 to BP7)
Cause attribution	Paired prime and composite controls separate “prime” from “remainder” and “odd”
Completeness claim	V(G) = 10 gives a finite, measurable set, and no suite with fewer than 10 tests can reach every independent path
Mathematical accuracy	The serial high-precision reference, invariants, and metamorphic relations define correctness independent of the code under test

Rounding versus real error	Integer-valued data and a derived tolerance distinguish genuine logic faults from floating-point reordering
System stability	Sanitizers, repetition, delay injection and memory-stress tests find races, overruns and uninitialized reads that only occur under unequal thread loads
Reproducibility	Deterministic sizes and logged seeds replace anonymous random runs
Regulatory suitability	Traceable coverage evidence, expected results, and tolerances support the documentation that medical device standards (for example IEC 62304) expect for software verification

Expected outcome:

Measure	Random-led	Structured
Branch coverage	Unknown	100% on the module
Basis paths	Unknown	10 of 10
Remainder and N < T cases	Rare	Guaranteed
Failure reproducibility	Low	Full, with seeds and sanitizer traces
Numeric correctness	Not checked	Checked against a reference and invariants
Race detection	By luck	Sanitizer plus repetition and delay injection
Total tests	Thousands, unmeasured	About 10 basis + 25 loop and size-class + 8 stability, plus seeded random as a supplement

6. Limits

- Basis path testing covers independent routes within the code, not every thread interleaving. Race behavior needs the sanitizer and stress layers, and even they give probabilistic assurance.
- Floating-point reproducibility across hardware (different vector units, fused multiply-add) can differ legitimately, so tolerances must be specified, not guessed.
- If primality really matters mathematically (for example, a transform whose algorithm changes for prime lengths, as in some FFT implementations that switch to a Rader or Bluestein method), that logic would add its own branches. The CFG should be rebuilt from the actual source, and the prime class tests become mandatory paths.
- For a clinical product, results also require validation against known reference images (phantoms), which no code-level test can replace.

7. Conclusion

Random testing failed because the defect depends on discrete code structure (an uneven remainder split, an odd-size correction, and N smaller than the thread count), and prime sizes are both sparse and special in how they interact with that structure. A control flow graph with V(G) = 10 yields ten independent paths, including the remainder and odd-size branches that prime inputs trigger, with composite controls to separate causes. Loop boundary tests, structure-aware size classes, a serial high-precision oracle with invariants, sanitizer and repetition-based stability tests, and mutation scoring then establish both mathematical accuracy and stability. Seeded random testing remains as a supplement that is logged and reproducible

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification